

Master of Science in Informatics at Grenoble
Master Informatique
Specialization Data science and Artificial intelligence (DSAI)

Updating Textual Descriptions from Evolving BPMN Models

David Cremer

July 24, 2025

Research project performed at the CONVECS team at INRIA

Under the supervision of:
Gwen Salaün

Defended before a jury composed of:
Silviu Maniu
Gwen Salaün
Mario Cortes-Cornax

Abstract

Keeping business process documentation consistent with updated process models is a common yet time-consuming challenge. When BPMN (Business Process Model and Notation) diagrams are changed, their associated textual descriptions often become outdated, leading to confusion and potential errors. This thesis presents a semi-automated method for updating such documentation through minimal, targeted edits. At the core of the approach are two strategies for detecting structural changes: the Longest Common Execution Sequence (LCES) algorithm, introduced in this work for acyclic and balanced models, and a custom beam search heuristic designed to handle unbalanced models and those containing cycles. These changes are used to construct structured prompts that guide a large language model in revising the original text. The method focuses on preserving validated portions of the documentation while ensuring semantic accuracy and stylistic consistency. The approach is implemented in a tool and evaluated through experiments, demonstrating its ability to produce reliable and coherent updates with reduced manual effort.

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my supervisor, Professor Gwen Salaün, for making this internship possible. His guidance, insightful feedback, and continuous support were invaluable throughout the duration of this work.

I am also sincerely thankful to Quentin Nivon, with whom I shared an office. Quentin was not only a constant source of advice and encouragement, but also always the first person I turned to for input—whether I was developing an idea or stuck on a problem. His help and presence greatly enriched my experience.

I would also like to thank Benjamin Dalmas, who acted as an additional supervisor during my internship. Our weekly discussions were instrumental in shaping my work, and his independent advice brought valuable new perspectives.

A warm thank you goes to the members of the CONVECS team at Inria, who welcomed me warmly and made me feel part of the group. I truly appreciated the conversations we shared during breaks, which created a pleasant and friendly atmosphere.

Finally, I would like to express my heartfelt thanks to my parents for their unwavering emotional support, especially during challenging times. I am also grateful to my friends and family for their presence, encouragement, and for always being there along the way. A special thanks to the person with whom I shared most of my days—your comforting presence and heartfelt support turned difficult days into manageable ones and good days into something even more. It meant more than I can say.

To all who contributed to this journey in ways big or small—thank you.

Résumé

Maintenir la cohérence entre la documentation d'un processus métier et sa représentation, mises à jour après mises à jour, est un défi courant, mais chronophage. Lorsqu'un diagramme BPMN est modifié, la documentation textuelle qui lui est associée devient généralement obsolète, et donc sujette à confusion ou erreur. Ce rapport présente une méthode semi-automatisée permettant de mettre à jour ladite documentation, par le biais

de modifications ciblées, et minimales. Le cœur de cette approche est constitué de deux stratégies de détection des modifications structurelles. La première est basée sur un algorithme de détection/calcul de la plus longue séquence d'exécution commune (LCES), introduit dans ce travail pour les modèles dits acycliques et équilibrés. La seconde est une heuristique personnalisée pour l'algorithme de recherche en faisceau (*beam search*), conçue pour traiter les modèles non-équilibrés ainsi que ceux contenant des cycles. Ces modifications structurelles sont ensuite utilisées pour construire des requêtes structurées, transmises à un grand modèle de langage (LLM) chargé de réviser la description originale. La méthode vise à préserver les portions déjà valides de la description tout en maintenant sa sémantique et une certaine cohérence stylistique. L'approche a été implémentée sous la forme d'un outil, puis évaluée au travers de diverses expérimentations, démontrant sa capacité à produire des mises à jour fiables et cohérentes de la description originale, tout en réduisant l'effort manuel.

Contents

Abstract	i
Acknowledgements	i
Résumé	i
1 Introduction	1
2 Preliminaries	5
2.1 BPMN	5
2.1.1 Balanced and Unbalanced BPMN Structures	6
2.2 Graph-based Representation of BPMN Models	7
2.2.1 Graph Edit Distance	9
Exact GED Computation via A*	9
Suboptimal GED via Heuristic-A*-Beamsearch	10
2.2.2 Longest Common Subsequence (LCS)	10
2.3 Large Language Models	11
3 Contributions	13
3.1 Overview	14
3.2 Longest Common Execution Sequence Approach	15
3.2.1 Process Reduction	17
3.2.2 Common Execution Subsequence (CES)	17
3.2.3 LCES Computation	19
3.2.4 LCES Computation Example	22
3.2.5 Complexity Analysis	25
3.2.6 Optimality of the LCES Algorithm	26
3.3 Beam Search Approach	27
3.3.1 Transformation Operations	29
3.3.2 Cost Model and Heuristic Function	30
Structural Edge Abstraction	31
Heuristic Function.	32
3.3.3 Practical Considerations	32
Heuristic Limitations	32

	Timeout Handling and Incomplete Search	33
	Beam Width and Search Trade-offs	33
3.4	GPT-Based Prompting Strategy	33
3.4.1	Prompting Methodology	35
	Full-Context Editing	35
	Natural-Language Dependency Encoding	36
	Behavioral Constraints and Role Framing.	36
	Localized Instruction Format.	36
	Stylistic and Semantic Constraints.	37
	Assumptions and Scope.	37
3.4.2	Task-Level Prompting	37
3.4.3	Sequence Flow Prompting	39
3.4.4	Gateway Prompting	41
4	Tool and Experiments	43
4.1	Tool	43
4.1.1	Overview	43
4.1.2	System Design	43
4.1.3	Implementation Notes	43
4.1.4	Scope and Limitations	44
4.2	Experiments	44
4.2.1	Experimental Setup	44
4.2.2	Manual evaluation criteria	45
4.2.3	Structural Alignment Metric	45
4.2.4	Metric Definition and Computation	46
4.2.5	Results	48
	Semantic Score	48
	Stylistic Score	49
	Structural Alignment Score (SAS)	49
	Execution Time	50
4.2.6	Discussion and Limitations	52
	Structural and Semantic Limitations	52
	SAS Behavior and Evaluation Limitations	52
	Performance and Execution Time	53
5	Related work	55
5.1	BPMN and UML Model Verbalization	55
5.2	Incremental and Update-Aware Documentation	56
5.3	Graph-to-Text Generation	56
5.4	Prompt-Based Language Models for Structured Input	57
5.5	Summary and Positioning	58
6	Conclusion	59
A	LCES Pseudocode	61
B	Tool Interface	65

Introduction

Business processes define how work is organized, coordinated, and executed within and across organizational boundaries. They encapsulate the operational logic, decision-making structures, and sequential dependencies that govern core functions such as order fulfillment, claims handling, or onboarding. As such, processes are not just execution blueprints—they are carriers of organizational intent, compliance requirements, and institutional knowledge. Effectively capturing, analyzing, and communicating these processes is essential for performance optimization, regulatory alignment, and cross-functional collaboration.

To support these goals, organizations typically rely on two complementary forms of process representation: graphical models and natural language descriptions. Graphical models, such as those expressed in Business Process Model and Notation (BPMN), offer a formalized and visual way to depict control-flow logic. They enable analysis, simulation, and execution of workflows, especially in business process management systems [7, 3]. However, these models are often inaccessible to non-technical stakeholders who rely instead on textual process documentation that conveys operational procedures, exceptions, and intent in human-readable form [14, 6]. In practice, textual representations are especially important during validation and communication tasks, where domain experts may lack modeling expertise but possess essential contextual knowledge [14]. Natural language descriptions serve not only as accessible documentation but also as critical tools for cross-functional alignment and verification [13].

Maintaining both representations—graphical and textual—is necessary but fraught with challenges. As processes evolve over time, BPMN models are frequently updated to reflect organizational changes, regulatory requirements, or optimization decisions. Unfortunately, accompanying textual documentation is rarely updated at the same pace, leading to inconsistencies between what is depicted and what is described [23, 3]. Such misalignment is not merely cosmetic: it can result in operational errors, compliance failures, and diminished trust in process artifacts [23]. Detecting and correcting these inconsistencies manually across large repositories is infeasible, especially as organizations scale to hundreds or thousands of processes [9]. Moreover, many companies already possess substantial unstructured documentation—manuals, policies, emails—that remains disconnected from formal models [9]. Ensuring consistency between models and text is therefore not only a matter of good documentation practice but a functional necessity in high-risk domains such as healthcare, finance, and manufacturing [13, 14].

Current industry practices for maintaining consistency are largely manual. Process analysts or domain experts are responsible for updating textual descriptions in response to model changes [13, 27]. This task is time-consuming, error-prone, and fundamentally unsustainable as the volume and complexity of process repositories grow. In large organizations, manual updating becomes impractical, especially given that textual descriptions often embed domain-specific terminology, implicit assump-

tions, and stylistic nuances that are not directly derivable from the control-flow logic of a BPMN model [17, 6]. Consequently, naive regeneration of text—while feasible with advanced natural language generation (NLG) techniques—often results in the loss of contextually rich and semantically significant information [9, 8].

While research has explored both directions of transformation—generating natural language from process models [13] and inferring models from textual requirements [19]—many existing approaches emphasize full regeneration rather than incremental update. This can lead to stylistic inconsistencies and reduced traceability, especially when only small changes occur in the underlying model. Moreover, regeneration approaches generally overlook the opportunity to reuse validated portions of existing documentation.

These observations give rise to a key insight: instead of regenerating textual descriptions from scratch, it is often more effective to treat the task as an edit-based revision problem—preserving what remains valid and updating only the elements affected by structural changes in the process model. This shift from generation to synchronization introduces new technical challenges: How can we detect and represent changes in a BPMN model in a structured and interpretable way? How can these changes be mapped to minimal, localized, and semantically aligned edits in the corresponding textual documentation [23, 14]? Addressing this gap is essential for reducing manual effort, increasing documentation reliability, and preserving institutional knowledge embedded in text.

To address this challenge, we propose a semi-automated approach that treats process documentation maintenance as a structured synchronization task between BPMN models and their corresponding textual descriptions. Instead of discarding the original text, we aim to preserve as much of it as possible by applying minimal, localized updates in response to model changes. This edit-based paradigm enables the preservation of domain-specific language, stylistic consistency, and contextual framing—all of which are frequently lost in approaches that rely on complete regeneration.

Our method combines structural analysis of BPMN models with prompt-based editing using large language models (LLMs). The central idea is to first identify the differences between the original and modified BPMN models in terms of their control-flow structure, and then use these differences to guide a series of natural language updates to the existing description. The result is a revised textual artifact that remains faithful to both the updated process logic and the original narrative voice.

To support this, we introduce a dual strategy for model comparison. For process models that are balanced and acyclic, we develop the Longest Common Execution Sequence (LCES) algorithm, which identifies the largest semantically consistent subset of shared execution paths between the original and updated models. This subset forms the stable foundation for preserving unchanged text, while deviations from it indicate where targeted edits are needed.

For more general cases—including models with loops, unbalanced gateways, or structural irregularities—we employ a custom beam search heuristic. This method approximates transformation sequences by iteratively exploring plausible edit operations, such as task insertions, deletions, and gateway modifications. Although not guaranteed to be optimal, this strategy is broadly applicable and effective in practice, even for structurally complex models.

Once the structural edits are determined, we convert them into natural language prompts that guide a large language model in performing corresponding text revisions. This prompt-based revision framework enables the LLM to perform controlled edits in a way that preserves style and fluency, ensuring that only the affected portions of the text are modified while leaving valid sections untouched.

By tackling the problem at the intersection of process modeling, graph transformation, and language generation, this work contributes to the growing body of research on semi-automated process documentation [13, 8, 27]. The proposed approach aims to reduce the manual effort involved in

maintaining textual descriptions, while ensuring that updates remain semantically aligned with the underlying process logic and faithful to the original narrative voice.

In what follows, we outline the structure of the thesis and detail the scope of each chapter.

Thesis Structure

The remainder of this thesis expands on the contributions outlined above, detailing the design, implementation, and evaluation of the proposed approach:

- **Chapter 2: Preliminaries** introduces the technical foundations, including BPMN syntax and semantics, graph-based process representations, edit distance computation, and the use of large language models in structured text editing.
- **Chapter 3: Contributions** elaborates on the core components of the method: the Longest Common Execution Sequence (LCES) algorithm for acyclic balanced models, the beam search heuristic for irregular structures, and the prompt design methodology for localized text revision.
- **Chapter 4: Tool and Experiments** describes the implemented system and experimental setup. It outlines the dataset, annotation criteria, and presents results assessing semantic fidelity and stylistic consistency of the generated updates.
- **Chapter 5: Related Work** positions this thesis in the broader research landscape, covering BPMN verbalization, graph-to-text generation, and prompt-based editing for structured data.
- **Chapter 6: Conclusion** summarizes the thesis and its edit-based approach to synchronizing BPMN models with text. It highlights key components, reflects on observed limitations, and outlines future work, including improved search strategies and prompt design.

Preliminaries

2.1 BPMN

BPMN 2.0 (BPMN, as a shorthand, in the rest of this paper) was published as an ISO/IEC standard in 2013 and is nowadays extensively used for modelling and developing business processes [11]. This paper focuses on activity diagrams including the BPMN constructs related to control-flow modelling and behavioural aspects. Specifically, the node types *event*, *task*, and *gateway*, and the edge type *sequence flow* are considered. Start and end events are used, respectively, to initialise and terminate processes. A task represents an atomic activity that has exactly one incoming and one outgoing flow. A sequence flow connects two nodes executed one after the other in a specific execution order. Gateways are used to control the divergence and the convergence of the execution flow. In this work, the two main kinds of gateways used in activity diagrams are considered, namely, *exclusive* and *parallel* gateways. Gateways with one incoming flow and multiple outgoing flows are called *splits*, e.g., split parallel gateway, while gateways with one outgoing flow and multiple incoming flows are called *merges*, e.g., merge parallel gateway. A parallel gateway creates concurrent executions for all its outgoing flows or synchronises concurrent executions of all its incoming flows. An exclusive gateway chooses one out of a set of mutually exclusive alternative incoming or outgoing flows. Such gateways can also be used to represent repetitive behaviours (i.e., loops). In this work, we limit our focus to the subset of BPMN syntax shown in Figure 2.1, which is sufficient to cover more than 90% of BPMN processes in practice [12].

In this work, a BPMN process is represented as a graph $G = (V, E, \lambda, \theta, v_{\text{start}}, v_{\text{end}})$, where a vertex

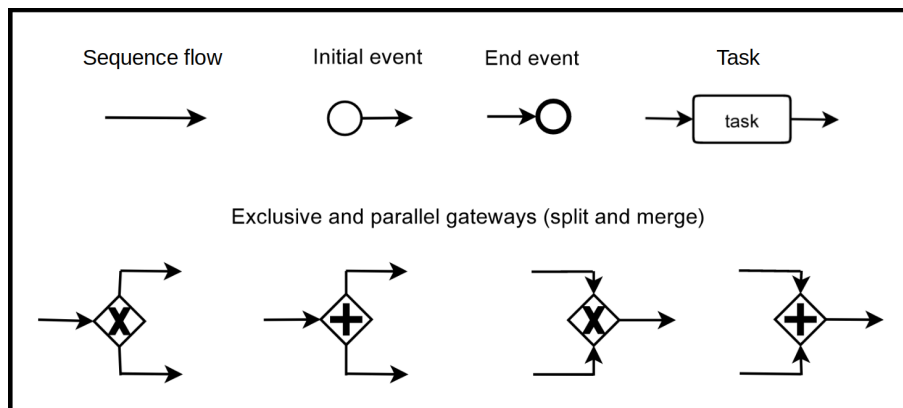


Figure 2.1: Supported fragment of the BPMN syntax

$v \in V$ is either an initial event, an end event, a task, or a gateway (exclusive or parallel, split or merge), and an edge $e \in E$ is a sequence flow. To stick to the usual representation of BPMN processes, vertices will be named *nodes* while edges will be named (*sequence*) *flows* in the rest of this work.

For simplicity, this work assumes that each BPMN process graph has exactly one start event and one end event, forming a single-entry, single-exit structure. While BPMN allows multiple start and end events, this assumption does not substantially limit generality. Models with multiple start events can be normalized by introducing a synthetic start event followed by a split gateway connecting to the original start points. Similarly, multiple end events can be unified by adding a synthetic merge gateway that joins all terminal branches to a single end event. These transformations preserve semantics while enabling consistent path analysis and simplifying the procedures described later in this thesis.

2.1.1 Balanced and Unbalanced BPMN Structures

A BPMN process is considered *balanced* when each split gateway has a corresponding merge gateway of the same type, such that all branches initiated by the split are explicitly synchronized at a single convergence point. This block-structured pattern is helpful for analysis and transformation, as it clearly delineates diverging and converging control-flow segments.

However, strict structural balancing is not required by the BPMN standard. A process is considered *unbalanced* if such one-to-one pairing between splits and merges is absent. In practice, many real-world BPMN models are unbalanced in this sense. For example, different outgoing flows of a split may be joined at different merge gateways, or multiple splits may lead to a shared merge. Importantly, unbalanced structures do not necessarily render a model invalid, as BPMN permits divergent paths to be joined at different points—or not at all—so long as the resulting process behavior is semantically coherent.

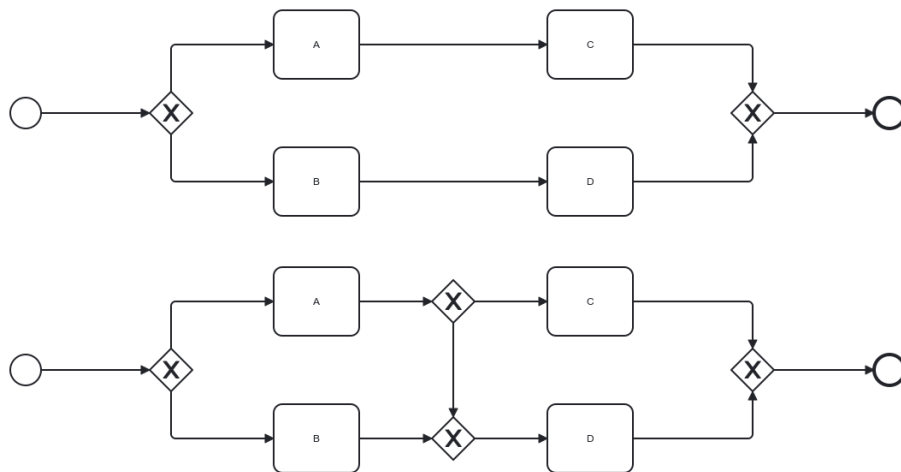


Figure 2.2: Comparison of balanced (top) and unbalanced (bottom) BPMN control-flow structures

Figure 2.2 illustrates this distinction. In the *balanced model* (top), all control-flow branches diverging from a split gateway are re-converged at a single, explicitly paired merge gateway, forming a strictly nested block structure. In contrast, the *unbalanced model* (bottom) joins branches at different merge gateways, without a one-to-one correspondence to the initial split. Both forms are semantically valid, but unbalanced structures may pose challenges for static analysis, comparison, or transformation techniques that rely on strict block structuring.

2.2 Graph-based Representation of BPMN Models

This work employs *directed, vertex-labelled graphs* as a formal representation of workflows, due to their ability to capture structural and control-flow relationships between process elements. To align with BPMN semantics, nodes are annotated with both name and type information, and each process is assumed to have a unique start and end event.

Definition 1 (Directed BPMN Graph). A directed BPMN graph is a tuple

$$G = (V, E, \lambda, \theta, v_{start}, v_{end}),$$

where:

- V is a finite set of nodes;
- $E \subseteq V \times V$ is a set of directed sequence flows;
- $\lambda : V \rightarrow \Lambda$ assigns each node a name label;
- $\theta : V \rightarrow T$ assigns each node a type, where

$$T = \{start, end, task, exclusive\ gateway, parallel\ gateway\};$$

- $v_{start} \in V$ is the unique start event;
- $v_{end} \in V$ is the unique end event.

A simplifying assumption adopted in this work is that each node $v \in V$ has a unique name label, i.e.,

$$\forall v_i, v_j \in V, v_i \neq v_j \Rightarrow \lambda(v_i) \neq \lambda(v_j).$$

Nodes are interchangeably referred to as *vertices*, and edges as *sequence flows*. In a directed graph, a node may have multiple *predecessors* or *successors*, forming paths and branches of execution.

Local structure. To reason about control-flow relationships between nodes in a BPMN graph, we introduce notation for their immediate neighbors in the graph:

Definition 2 (SUCCESSORS). Let $G = (V, E, \lambda, \theta, v_{start}, v_{end})$ be a directed BPMN graph. For any node $v \in V$, we define

$$\text{SUCCESSORS}(v) := \{u \in V \mid (v, u) \in E\}$$

as the set of immediate successors of v in G .

Definition 3 (PREDECESSORS). Let $G = (V, E, \lambda, \theta, v_{start}, v_{end})$ be a directed BPMN graph. For any node $v \in V$, we define

$$\text{PREDECESSORS}(v) := \{u \in V \mid (u, v) \in E\}$$

as the set of immediate predecessors of v in G .

These functions allow us to formally describe the local control-flow context of a node in terms of its incoming and outgoing sequence flows.

Paths and reachability. Based on this local structure, we define longer-range relationships:

Definition 4 (Path). Let $G = (V, E, \lambda, \theta, v_{start}, v_{end})$ be a directed BPMN graph. A path in G is a sequence of nodes $p_G = \langle v_1, v_2, \dots, v_n \rangle$ such that for all $i \in \{1, \dots, n-1\}$, $(v_i, v_{i+1}) \in E$.

We denote:

- $\mathcal{P}_G$ as the set of all paths in G ;
- $\mathcal{P}_G(v)$ as the set of all paths starting at node v ;
- $\mathcal{P}_G(u, v)$ as the set of all paths from node u to node v , i.e.,

$$\mathcal{P}_G(u, v) := \{ \langle v_1, \dots, v_n \rangle \in \mathcal{P}_G \mid v_1 = u, v_n = v \}.$$

A node v is said to be *reachable* from a node u (denoted $u \xrightarrow{G} v$) if $\mathcal{P}_G(u, v) \neq \emptyset$.

Gateway roles. Gateways in BPMN serve to either diverge or converge control flow. Based on their immediate predecessors and successors, we distinguish two structural roles:

- A node v is a *split gateway* if $|\text{PREDECESSORS}(v)| = 1$ and $|\text{SUCCESSORS}(v)| > 1$;
- A node v is a *merge gateway* if $|\text{PREDECESSORS}(v)| > 1$ and $|\text{SUCCESSORS}(v)| = 1$.

To refer to these roles more precisely in formal statements, we define boolean predicates:

$$\text{ISGATEWAY}(v) := \theta(v) \in \{ \text{exclusive gateway}, \text{parallel gateway} \},$$

$$\text{ISSPLIT}(v) := \text{ISGATEWAY}(v) \wedge |\text{PREDECESSORS}(v)| = 1 \wedge |\text{SUCCESSORS}(v)| > 1,$$

$$\text{ISMERGE}(v) := \text{ISGATEWAY}(v) \wedge |\text{PREDECESSORS}(v)| > 1 \wedge |\text{SUCCESSORS}(v)| = 1.$$

Balanced structure. While BPMN permits arbitrary combinations of splits and merges, many process models follow a block-structured style in which each split has a uniquely corresponding merge. This regularity is not required for correctness, but simplifies structural reasoning and enables certain graph transformations. We formalize this pattern as follows:

Definition 5 (Balanced Graph). A BPMN graph $G = (V, E, \lambda, \theta, v_{start}, v_{end})$ is balanced if every split gateway $g_s \in V$ has a unique corresponding merge gateway $g_m \in V$ such that the following conditions hold.

Let the split–merge region associated with the pair (g_s, g_m) be defined as:

$$R(g_s, g_m) := \{ v \in V \mid \exists p \in \mathcal{P}_G(g_s, g_m) \text{ such that } v \in p \}.$$

Then:

1. Every path $p \in \mathcal{P}_G(g_s)$ that leaves the region passes through g_m exactly once;
2. The merge gateway g_m is not reachable from any node outside the region, except via a path that passes through g_s .

In some cases, we restrict attention to *acyclic* balanced graphs, where the graph and especially the split–merge region $R(g_s, g_m)$ contains no cycles. This simplification supports well-defined branching structures and facilitates static analysis.

In the case of a balanced graph, we can define a function that returns the corresponding merge for a given split:

Definition 6 (Corresponding Merge). *If G is a balanced BPMN graph, we define a partial function*

$$\text{MERGE} : V \rightharpoonup V$$

such that for each split gateway g_s , $\text{MERGE}(g_s)$ returns the unique merge g_m that completes the split–merge pair (g_s, g_m) .

Within a balanced split–merge region, the execution diverges from the split gateway g_s along a set of independent control-flow paths and converges again at the merge gateway g_m . Each such path represents a distinct *branch* of the region.

Definition 7 (Branch). *Let (g_s, g_m) be a balanced split–merge pair in a balanced and acyclic BPMN graph G , and let $R(g_s, g_m)$ be the corresponding split–merge region.*

A branch of this region is any path $p = \langle u_1, u_2, \dots, u_k \rangle$ such that:

- $u_1 \in \text{SUCCESSORS}(g_s)$;
- $(u_k, g_m) \in E$, i.e., the path ends immediately before g_m ;
- $\forall i \in \{1, \dots, k\}, u_i \in R(g_s, g_m) \setminus \{g_m\}$.

The set of all such branches is denoted $\mathcal{B}(g_s)$.

2.2.1 Graph Edit Distance

The *Graph Edit Distance* (GED) is a flexible method for quantifying structural differences between graphs based on edit operations. GED is defined as the minimum-cost sequence of edit operations required to transform one graph into another. The allowable operations typically include insertion, deletion, and substitution of vertices and edges. Each operation is assigned a cost, and the total cost of an edit path reflects the degree of structural difference between the two graphs.

Definition 8 (Graph Edit Distance). *Let G_1 and G_2 be two graphs. The Graph Edit Distance (GED) between G_1 and G_2 , denoted $\text{GED}(G_1, G_2)$, is defined as:*

$$\text{GED}(G_1, G_2) := \min_{S \in \mathcal{S}} \sum_{o \in S} \text{cost}(o),$$

where $\mathcal{S}$ is the set of all valid edit paths transforming G_1 into G_2 , and each $o \in S$ denotes a single edit operation with an associated non-negative cost $\text{cost}(o)$.

GED is highly flexible and applicable to various types of attributed and unconstrained graphs, making it a powerful tool in structural pattern recognition and related domains. In this context, the unique vertex labels assumed earlier simplify the substitution process by reducing ambiguity.

Exact GED Computation via A*

The computation of GED is known to be NP-hard due to the combinatorial number of possible edit paths between graphs. One of the most widely used algorithms for computing the exact GED is the A* search algorithm [18]. A* represents the search space as a tree of partial edit paths, exploring it in a best-first manner guided by a cost function:

$$f(p) = g(p) + h(p)$$

Here, $g(p)$ is the cost accumulated so far, and $h(p)$ is a heuristic estimate of the remaining cost to complete the transformation. If the heuristic $h(p)$ is admissible (i.e., it never overestimates the true cost), the A* algorithm is guaranteed to find the optimal path.

Despite this theoretical guarantee, A* becomes infeasible for graphs beyond a modest size due to its exponential time and space complexity.

Suboptimal GED via Heuristic-A*-Beamsearch

To mitigate the computational burden of exact GED computation, Neuhaus et al. [18] proposed a suboptimal variant based on *beam search*. This method, referred to as A*-*beamsearch*, restricts the number of partial edit paths retained in the search frontier (OPEN set) to a fixed beam width s . At each iteration, only the s most promising paths—those with the lowest estimated total cost

$$f(p) = g(p) + h(p)$$

are retained, and the rest are discarded.

This strategy significantly reduces the breadth of the search tree, achieving substantial gains in both runtime and memory efficiency. Although A*-beamsearch does not guarantee the optimal edit path, the approximated distance is often sufficiently accurate for practical applications, particularly when working with large graphs.

In practice, choosing an appropriate beam width s allows for a trade-off between computational speed and matching precision. Smaller values of s yield faster computations at the cost of higher approximation error, while larger values maintain higher accuracy with increased runtime.

2.2.2 Longest Common Subsequence (LCS)

The *Longest Common Subsequence (LCS)* problem is a classical problem in string comparison and pattern matching. Given two finite sequences over a common alphabet, the goal is to find the longest sequence that appears in both inputs in the same relative order (but not necessarily contiguously).

Definition 9 (Longest Common Subsequence). *Let $A = \langle a_1, \dots, a_m \rangle$ and $B = \langle b_1, \dots, b_n \rangle$ be sequences over an alphabet Σ . A sequence $C = \langle c_1, \dots, c_k \rangle$ is a common subsequence of A and B if:*

$$C \sqsubseteq A \quad \text{and} \quad C \sqsubseteq B,$$

where $\sqsubseteq$ denotes subsequence inclusion. The Longest Common Subsequence (LCS) is any common subsequence of maximal length k .

The classical solution to the LCS problem is a dynamic programming approach based on the following recurrence [10]:

$$\text{LCS}(i, j) = \begin{cases} 1 + \text{LCS}(i-1, j-1) & \text{if } a_i = b_j, \\ \max(\text{LCS}(i-1, j), \text{LCS}(i, j-1)) & \text{otherwise.} \end{cases}$$

This recurrence fills a matrix of size $(m+1) \times (n+1)$, where each entry represents the length of the longest common subsequence of the prefixes $A[1..i]$ and $B[1..j]$. The final cell $\text{LCS}(m, n)$ contains the length of the overall LCS.

In addition to computing the length, this algorithm can be extended to reconstruct the actual subsequence by tracing the dynamic programming matrix from bottom-right to top-left.

Hirschberg [10] further showed that both the LCS length and the subsequence itself can be computed in *linear space*, enabling efficient applications in settings with large input sequences.

2.3 Large Language Models

Large Language Models (LLMs) are pre-trained neural networks capable of understanding and generating human-like text. They are widely used in natural language processing tasks such as text classification, summarization, question answering, and structured data extraction.

In this work, the GPT-4.1 model—developed by OpenAI—is used via the OpenAI API to support the processing and transformation of textual descriptions of business processes. GPT, which stands for *Generative Pre-trained Transformer*, is a state-of-the-art LLM that operates through prompt-based interactions. In this setup, the model receives carefully constructed natural language prompts that guide it to perform specific tasks such as task deletion, dependency modification, or reformulation of process descriptions.

The use of GPT in this thesis is not generative in the creative sense, but rather controlled and deterministic: prompts are algorithmically constructed to apply well-defined changes while preserving the semantics of the original process. This approach involves a technique known as *prompt engineering*, which refers to the design and formulation of input prompts that guide the model's behavior. Prompts are crafted to include task-specific instructions, structural constraints, and transformation goals, ensuring the model performs deterministic edits rather than open-ended generation. A detailed description of the prompting strategy is provided in Section 3.4.

Contributions

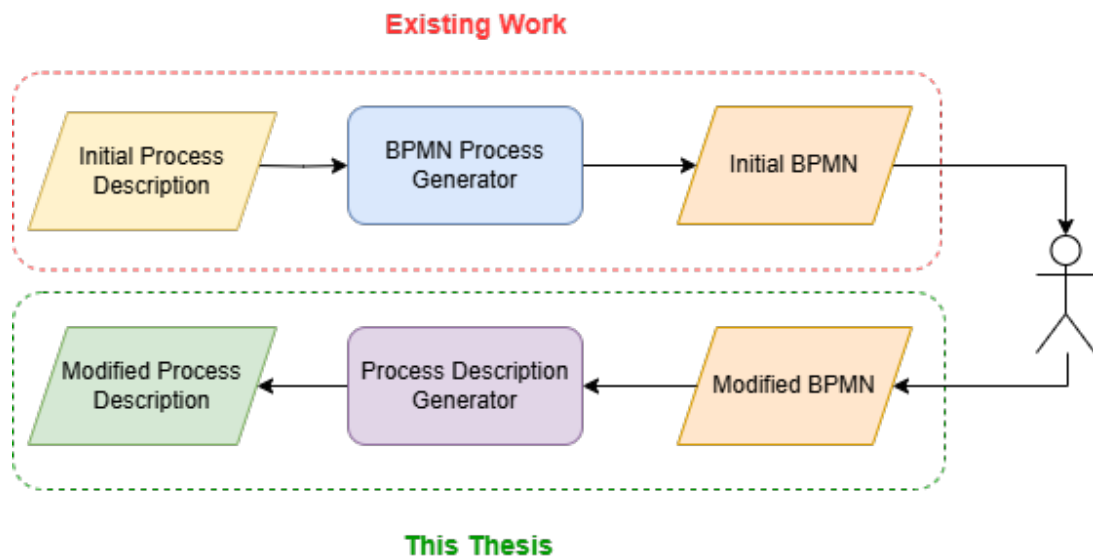


Figure 3.1: Overview of the process description update task

This thesis focuses on the generation of textual process descriptions from BPMN (Business Process Model and Notation) models, specifically in the context of updating a description after the model has been modified. As illustrated in Figure 3.1, the process typically begins with a textual description from which a BPMN model is created, either manually or with tool support, as proposed by Nivon et al. [19]. This BPMN model may then be modified by a user, for example by adding or removing tasks, or changing the control flow.

The contribution of this work begins after such modifications. The goal is to generate an updated textual description that reflects the changes made to the BPMN model. While generating a new description from a BPMN model is relatively straightforward—using, for instance, a traversal of the process structure combined with modular text generation—this thesis addresses a more specific and challenging task: producing an updated description that stays as close as possible to the original in both wording and structure.

In particular, parts of the BPMN model that were not modified should result in unchanged or nearly unchanged text. This requires careful handling to preserve the style and phrasing of the original description, while still accurately reflecting the changes in the process. Furthermore, any newly

generated text—corresponding to modified or added parts of the model—should be written in a style consistent with the original, ensuring a cohesive and natural integration into the existing description.

By focusing on this goal, the thesis contributes a method for generating updated process descriptions that are not only correct but also consistent with the original version—helping to improve the clarity, maintainability, and usability of process documentation.

3.1 Overview

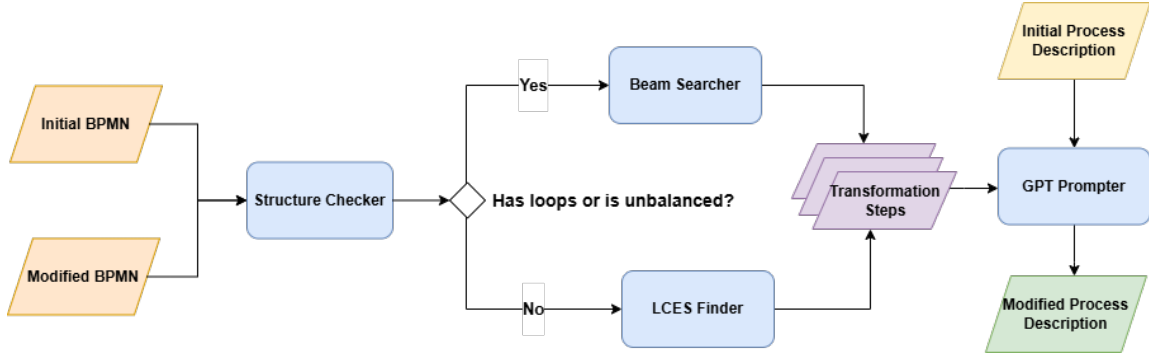


Figure 3.2: High-level workflow of the transformation method

The central idea of the proposed approach is to transform an initial process description into an updated version that accurately reflects changes made to its corresponding BPMN model. Rather than generating a new description from scratch, our method aims to preserve as much of the original wording and structure as possible. To achieve this, we decompose the change between the initial and modified BPMN models into a sequence of small, incremental transformation steps. These steps serve as the backbone for updating the textual description: for each transformation applied to the model, a corresponding prompt is used to adapt the description in a controlled and localized manner. The intermediate models resulting from these steps are valid or near-valid BPMN models, possibly with minor structural relaxations, ensuring they remain interpretable and useful for guiding the text update process.

This strategy enables a *stepwise update process*, where the original description is progressively transformed to match the modified model. Crucially, this incremental process preserves unmodified portions of the description and ensures that newly generated segments blend stylistically with the original.

As illustrated in Figure 3.2, the first stage of the pipeline analyzes the structural properties of the initial and modified BPMN models. Based on this analysis, we select an appropriate method for computing the transformation steps. These steps effectively capture the minimal set of changes required to convert the initial model into the modified one.

If the models are acyclic and structurally balanced, we apply the *Longest Common Execution Sequence (LCES)* algorithm. This method identifies shared execution paths and isolates the changes, resulting in an efficient and near-optimal transformation sequence. The LCES approach is reliable in this context because its assumptions hold true for balanced, loop-free graphs.

For all other cases—i.e., when the graph is unbalanced or contains loops—we apply a *beam search* strategy. This heuristic search does not guarantee optimality nor convergence but is applicable to arbitrary BPMN models and often produces satisfactory results in practice. It incrementally explores alternative transformation sequences and selects the most promising ones based on a scoring function, allowing it to handle the increased complexity of general process structures.

While LCES produces strictly valid intermediate models at each step, the beam search may temporarily operate on structurally imperfect models, which are corrected by the end of the transformation.

The output of either method is a sequence of transformation steps, each representing a localized structural change to the BPMN model. These steps are then used to guide the GPT-based update of the textual description. Starting from the original text, GPT receives prompts that correspond to individual transformation steps, allowing it to incrementally revise the description in alignment with the evolving process model.

By combining fine-grained model analysis with targeted language generation, this method ensures that the updated description remains faithful to both the modified process and the original narrative. This approach not only enhances the accuracy and consistency of process documentation but also improves its maintainability by minimizing unnecessary rewrites.

3.2 Longest Common Execution Sequence Approach

When both the initial and modified BPMN models are *acyclic* and *balanced*, we apply a *Longest Common Execution Sequence (LCES)* approach to derive a sequence of transformation steps. The LCES identifies a structurally aligned execution path shared between the two models, serving as a behavioral reference. From this shared core, we compute a minimal set of structural changes—specifically, node deletions, gateway type changes, and additions—that together describe how to transform one model into the other. These transformation steps form the basis for incrementally updating the corresponding textual process description in a way that closely mirrors the underlying changes.

A notable property of this approach is that each intermediate BPMN model remains structurally valid and connected throughout the transformation. This ensures that every step corresponds to a coherent process state, making it well-suited for driving interpretable and consistent text updates.

It is important to note that our goal is not to compute an exact graph-theoretic match, but rather to capture a maximal set of behaviorally aligned execution elements. This is **not a classical Maximum Common Subgraph (MCS)** problem, which seeks structural isomorphism and is NP-hard [5]. Instead, our approach is inspired by the **Longest Common Subsequence (LCS)** problem, which focuses on preserving the relative order and semantic alignment of matched elements, rather than strict structural equivalence.

While LCS is traditionally defined over linear sequences, the concept generalizes naturally to process models represented as directed acyclic graphs. In this setting, execution order is captured via control-flow dependencies, and elements such as tasks, events, and gateways can be treated as partially ordered components of an abstract execution sequence. By identifying the longest such sequence that is shared between two models—preserving both semantic node equivalence and relative ordering—we can extract a meaningful common structure even in the presence of structural divergence, such as branching or parallelism.

A central feature of the LCES approach is its hierarchical handling of *balanced split-merge blocks*. For each such gateway pair, the algorithm treats the outgoing branches as independent regions and attempts pairwise alignment within them. When a consistent alignment is found, the block is considered *flattened* for alignment purposes, allowing the LCES computation to proceed as if the control structure were linear—while still preserving the original branching semantics. Rather than abstracting the block into a single node, the algorithm preserves its internal structure but aligns it as a flat sequence whenever branch-level consistency allows.

The aim of this method is to maximize the execution overlap between the two process models, minimizing the number of unmatched elements. Reducing these discrepancies in turn minimizes the

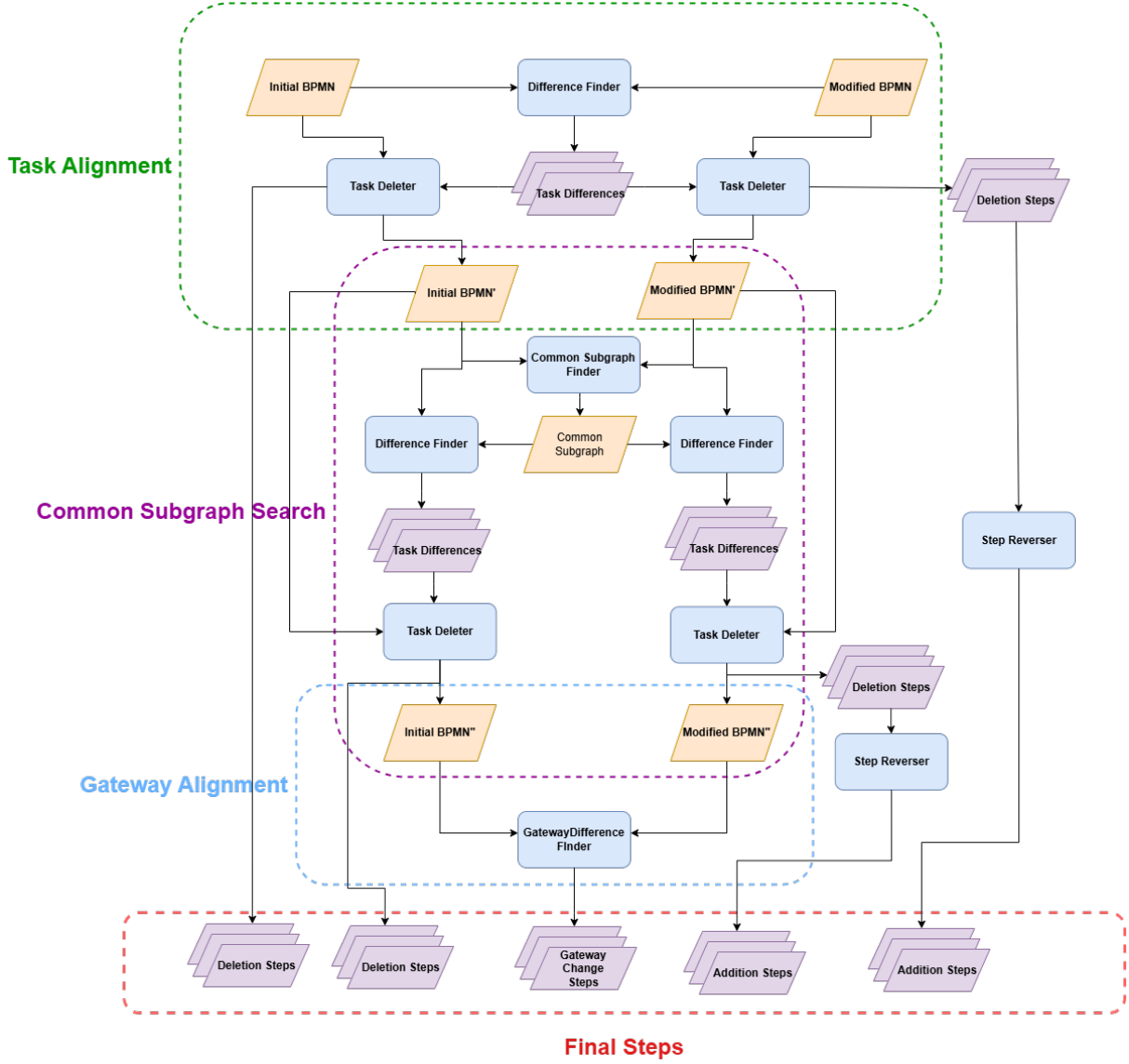


Figure 3.3: Overview of the LCES-based transformation procedure

number of transformation steps required—and consequently, the number of GPT prompting steps needed to incrementally update the accompanying process description.

The full transformation procedure is illustrated in Figure 3.3. It begins with a *task alignment phase*, in which tasks that are not part of the shared execution core are removed from both models. As these elements must eventually be inserted or deleted, excluding them early reduces the complexity of the subsequent computation. This results in a pair of simplified models, denoted *Initial BPMN'* and *Modified BPMN'*.

These are passed to the *LCES computation phase*, which identifies the longest common execution subsequence. The output is a structurally aligned pair of models, *Initial BPMN''* and *Modified BPMN''*, in which only gateway type mismatches may remain.

The final step is *gateway alignment*, where gateway types in *Initial BPMN''* are adjusted to match those in *Modified BPMN''*, ensuring consistent control-flow semantics.

The transformation path is then constructed based on the differences between the original models

and the shared LCES core. First, we derive a set of *deletion steps* by identifying elements present in the initial model but not in the LCES. Second, we compute *gateway type change steps* to resolve structural mismatches in branching logic. Finally, we derive *addition steps* by identifying elements in the modified model that are absent from the LCES. These additions effectively reverse the deletions when applying the transformation in the forward direction.

Together, these three stages—deletions, gateway changes, and additions—form a complete and ordered *transformation path* from the initial to the modified model. This path, shown at the bottom of Figure 3.3, serves as the foundation for sequential GPT prompting, enabling localized and interpretable updates to the textual process description.

3.2.1 Process Reduction

To improve structural alignment between semantically equivalent BPMN models, we apply a reduction step prior to computing the common execution subsequence. This transformation removes gateway constructs that do not alter control-flow behavior but may introduce syntactic variation—such as directly nested splits of the same type with nested merges, or gateways that do not perform any actual routing.

Definition 10 (Reduced Graph). *Let $G = (V, E, \lambda, \theta, v_{start}, v_{end})$ be a balanced BPMN graph.*

We say that G is reduced if it satisfies the following two conditions:

1. *No directly nested splits of the same type with nested merges:*

$$\neg \exists (u, v) \in E \text{ such that } \text{ISSPLIT}(u) \wedge \text{ISSPLIT}(v) \wedge \theta(u) = \theta(v) \wedge (\text{MERGE}(v), \text{MERGE}(u)) \in E.$$

That is, a split gateway must not immediately lead to another split of the same type if their corresponding merges are also nested. Such patterns are structurally redundant and can be flattened.

2. *No non-branching gateways:*

$$\forall g \in V, \theta(g) \in \{\text{exclusive gateway}, \text{parallel gateway}\} \Rightarrow \text{ISSPLIT}(g) \vee \text{ISMERGE}(g).$$

That is, every gateway must function as a divergence or convergence point. Gateways with exactly one incoming and one outgoing sequence flow are considered semantically neutral and are removed.

While reduction is not strictly required for the correctness of the LCES algorithm, it serves as a valuable preprocessing step. By eliminating syntactic artifacts that do not affect execution behavior, reduction improves the structural alignment of models and can increase the size of the resulting common execution subsequence.

3.2.2 Common Execution Subsequence (CES)

To reason about structural similarity between BPMN models, we define a notion of behavioral alignment based on matched pairs of semantically equivalent nodes. This alignment must preserve control-flow consistency and respect the structure of constructs such as branching and merging.

The foundation of this alignment is a binary node matching relation that captures semantic compatibility: nodes are considered equivalent if they play the same role (e.g., start events, end events, tasks with identical labels, or gateways with the same behavioral function).

We begin by formalizing this node-level equivalence before defining the global structure of a common execution subsequence.

Definition 11 (Node Match). *Let $G = (V, E, \lambda, \theta, v_{start}, v_{end})$ and $G' = (V', E', \lambda', \theta', v'_{start}, v'_{end})$ be BPMN process graphs.*

We define the binary relation

$$\text{MATCH} \subseteq V \times V'$$

by setting:

$$\text{MATCH}(v, v') \iff \left(\begin{array}{l} \text{ISSPLIT}(v) \wedge \text{ISSPLIT}(v') \\ \vee \text{ISMERGE}(v) \wedge \text{ISMERGE}(v') \\ \vee \theta(v) = \theta'(v') = \text{task} \wedge \lambda(v) = \lambda'(v') \\ \vee \theta(v) = \theta'(v') = \text{start} \\ \vee \theta(v) = \theta'(v') = \text{end} \end{array} \right)$$

With node matching and reachability defined, we can now formalize what constitutes a *common execution subsequence* (CES) between two BPMN models. Intuitively, a CES consists of matched pairs of semantically equivalent nodes that appear in compatible control-flow positions: if one pair precedes another in one model, it must do so in the other as well. Each node may appear at most once, and matched elements must respect the structure of parallel control flow.

This formulation builds directly on reachability and the notion of balanced split–merge regions. No graph transformations are required; instead, we ensure structural soundness by checking that matches occur within coherent branches and that gateway pairs are properly scoped.

Definition 12 (Common Execution Subsequence (CES)). *Let $G = (V, E, \lambda, \theta, v_{start}, v_{end})$ and $G' = (V', E', \lambda', \theta', v'_{start}, v'_{end})$ be two acyclic balanced BPMN graphs.*

A common execution subsequence (CES) is a sequence of matched node pairs

$$\Pi = \langle (v_1, v'_1), (v_2, v'_2), \dots, (v_k, v'_k) \rangle \subseteq V \times V'$$

such that the following conditions hold:

1. **Matching:** *Each pair (v_i, v'_i) satisfies:*

$$\text{MATCH}(v_i, v'_i) \quad \text{for all } i \in \{1, \dots, k\}.$$

2. **Uniqueness:** *No node appears in more than one pair:*

$$v_i \neq v_j \quad \text{and} \quad v'_i \neq v'_j \quad \text{for all } i \neq j.$$

3. **Order consistency:** *For all $i < j$, the relative execution order of matched nodes must agree:*

$$v_i \xrightarrow{G} v_j \iff v'_i \xrightarrow{G'} v'_j.$$

4. **Forward compatibility:** *For all $i < j$, the CES must not contradict control flow:*

$$v_j \not\xrightarrow{G} v_i \quad \text{and} \quad v'_j \not\xrightarrow{G'} v'_i.$$

5. **Branch coherence:** Let $\mathcal{M} \subseteq V \times V$ and $\mathcal{M}' \subseteq V' \times V'$ be the sets of split–merge pairs in G and G' , respectively. For each region $(g_s, g_m) \in \mathcal{M}$ and its corresponding $(g'_s, g'_m) \in \mathcal{M}'$, define:

$$\Pi_{(g_s, g_m)} := \{v \in V \mid (v, v') \in \Pi, v \in R(g_s, g_m)\}$$

$$\Pi'_{(g'_s, g'_m)} := \{v' \in V' \mid (v, v') \in \Pi, v' \in R(g'_s, g'_m)\}$$

Then one of the following must hold:

- Both gateways (g_s, g'_s) and (g_m, g'_m) are in Π ;
- All nodes in $\Pi_{(g_s, g_m)}$ lie within a single branch $b \in \mathcal{B}(g_s)$, and all nodes in $\Pi'_{(g'_s, g'_m)}$ lie within a single branch $b' \in \mathcal{B}(g'_s)$.

This formalization provides the basis for computing behaviorally aligned regions in structurally divergent models. The next section introduces a recursive procedure to compute such sequences based on reachability, matching, and branch coherence.

3.2.3 LCES Computation

The Longest Common Execution Sequence (LCES) computation generalizes the classical Longest Common Subsequence (LCS) algorithm to the domain of process models, which are represented as directed acyclic graphs (DAGs) in the BPMN formalism. While LCS is defined over linear sequences, process models capture richer control-flow constructs such as branching, merging, and parallelism. In this setting, sequential order is replaced by graph reachability, and alignment must preserve both semantic meaning and structural consistency across complex execution paths.

To compute the LCES, we define a recursive procedure that operates over matched node pairs in the two models. Let $v \in V$ and $v' \in V'$ be nodes in the respective BPMN graphs. The function $\text{LCES}(v, v')$ returns the *set of all longest common execution subsequences* starting from the pair (v, v') . If $\text{MATCH}(v, v')$ holds, the pair is included in each result, and the computation proceeds by recursively exploring all valid successor combinations. If the nodes do not match, the function advances independently in either model and returns all longest results from those branches.

This recurrence mirrors the structure of the classical LCS algorithm (cf. Section 2.2.2), which operates over two linear sequences. In the standard formulation, the algorithm recursively evaluates three options at each position: match, skip-left, or skip-right. The recurrence is typically expressed as:

$$\text{LCS}(i, j) = \begin{cases} 1 + \text{LCS}(i+1, j+1) & \text{if } x_i = y_j, \\ \max(\text{LCS}(i+1, j), \text{LCS}(i, j+1)) & \text{otherwise.} \end{cases}$$

This recurrence mirrors the classical LCS recurrence (cf. Section 2.2.2), but is expressed using *forward indexing*—that is, using $i+1$ and $j+1$ instead of $i-1$ and $j-1$ —to align with the traversal direction used in our graph-based formulation. Both forms are mathematically equivalent, differing only in the direction of recursion.

In our graph-based generalization, the same high-level structure applies: we consider matching the current pair, or skipping a node in either model. However, the branching nature of process models introduces multiple successors for each node, which greatly increases the number of possible continuations. As a result, the LCES recurrence operates over partial orders rather than sequences, and must evaluate a much larger space of possible alignments.

This formulation serves as the basis for our generalization to the Longest Common Execution Sequence (LCES), which extends classical sequence alignment to settings where control flow is governed by concurrency and branching. The resulting approach captures behaviorally aligned execution paths even when process models differ significantly in structure.

Definition 13 (LCES Recurrence for DAGs). *Let $G = (V, E, \lambda, \theta)$ and $G' = (V', E', \lambda', \theta')$ be acyclic balanced BPMN graphs. Define $\text{LCES}(v, v')$ as the set of all longest common execution subsequences starting from nodes $v \in V$ and $v' \in V'$. Then:*

$$\text{LCES}(v, v') := \left\{ \pi \in \mathcal{C}(v, v') \mid |\pi| = \max_{\pi' \in \mathcal{C}(v, v')} |\pi'| \right\}$$

where the set of candidate continuations $\mathcal{C}(v, v')$ is defined as:

$$\mathcal{C}(v, v') := \left\{ \begin{array}{ll} \{(v, v')\} \cup \pi & \text{for } \pi \in \text{EXTEND}(v, v') \quad \text{if } \text{MATCH}(v, v') \\ \pi & \text{for } \pi \in \bigcup_{v'' \in \text{NEXT}(v)} \text{LCES}(v'', v') \quad (\text{skip } v, \text{ always considered}) \\ \pi & \text{for } \pi \in \bigcup_{v'' \in \text{NEXT}(v')} \text{LCES}(v, v'') \quad (\text{skip } v', \text{ always considered}) \end{array} \right\}$$

The function $\text{EXTEND}(v, v')$ is defined as:

$$\left\{ \begin{array}{ll} \text{FLATTEN}(\{\text{LCES}(u, u') \mid (u, u') \in \text{NEXT}(v) \times \text{NEXT}(v')\}) & \text{if } \text{ISPLIT}(v) \wedge \text{ISPLIT}(v') \\ \bigcup \{\text{LCES}(u, u') \mid (u, u') \in \text{NEXT}(v) \times \text{NEXT}(v')\} & \text{otherwise} \end{array} \right\}$$

Note that the outer maximization in the LCES recurrence applies to *all* branches, including the matched case. This differs subtly from the classical LCS recurrence, where the max is only evaluated when symbols do not match. In the graph-based setting of LCES, including a matching node pair (v, v') does not necessarily yield the longest possible execution sequence. For example, it may be more beneficial to bypass the current pair to align structurally corresponding gateways later in the graph—such as matching a split gateway in one model with a semantically equivalent split further ahead in the other. To account for this, the recurrence always evaluates all three possibilities—match, skip v , and skip v' —and collects all continuations of maximal length.

When both v and v' are matched split gateways, a more specialized alignment strategy is used. Simple recursive descent is insufficient to capture structurally coherent paths across divergent control-flow branches. Instead, the algorithm computes all pairwise LCES alignments between the outgoing branches of the splits. These branch-wise results are then passed to a *flattening operation*, which extracts a semantically consistent region spanning from the matched split gateways to their corresponding merge gateways.

However, it is possible that a complete alignment along a single branch—bypassing the split and merge gateways entirely—yields a longer execution subsequence. This scenario is handled by a dedicated fallback mechanism, which replaces the flattened region if an unstructured branch-level alignment yields a strictly longer common subsequence. The formal conditions for this replacement are defined in Definition 15.

Let $g_s \in G$ and $g'_s \in G'$ be matched split gateways, and let $g_m \in G$, $g'_m \in G'$ be their corresponding merge gateways. The outgoing branches from these splits (as defined in Definition 7) form the sets:

$$\mathcal{B}(g_s) = \{B_1, \dots, B_m\}, \quad \mathcal{B}(g'_s) = \{B'_1, \dots, B'_n\}.$$

To construct the flattened region between the split and merge gateways, we compute local LCES alignments for each pair of branches. For each pair (B_i, B'_j) , we identify the respective entry nodes $u \in B_i$ and $u' \in B'_j$, and compute a branch-level alignment:

$$\pi_{ij} := \text{LCES}(u, u').$$

Only if this alignment contains the matched merge nodes (g_m, g'_m) , the sequence is split into two parts: a prefix π_{ij}^{pre} , ending at the merge nodes, and a suffix π_{ij}^{post} , continuing from the merge nodes onward.

Each valid alignment is collected into the set:

$$\mathcal{M} = \left\{ (\pi_{ij}^{\text{pre}}, \pi_{ij}^{\text{post}}, B_i, B'_j) \mid (g_m, g'_m) \in \pi_{ij} \right\}.$$

To identify the most consistent alignment across branches, we extract the maximally aligned subset of pre-merge sequences:

Definition 14 (Maximally Aligned Branches). *Let $\mathcal{M}$ be the set of valid branch alignments, where each element is a tuple*

$$(\pi^{\text{pre}}, \pi^{\text{post}}, B_i, B'_j).$$

The maximally aligned subset $\mathcal{M}_{\text{max}} \subseteq \mathcal{M}$ is defined as:

$$\mathcal{M}_{\text{max}} := \underset{\substack{\mathcal{S} \subseteq \mathcal{M} \\ \text{all } B_i \text{ and } B'_j \text{ pairwise distinct}}}{\text{argmax}} \sum_{(\pi^{\text{pre}}, \pi^{\text{post}}, B_i, B'_j) \in \mathcal{S}} |\pi^{\text{pre}}|.$$

Next, we select the longest post-merge continuation among all alignments in $\mathcal{M}$:

$$\pi_{\text{post}}^* := \underset{(\pi^{\text{pre}}, \pi^{\text{post}}, B_i, B'_j) \in \mathcal{M}}{\text{argmax}} |\pi^{\text{post}}|.$$

The complete flattened region spanning from the split gateways to the merge gateways is then defined as:

$$F(g_s, g'_s) := \langle (g_s, g'_s) \rangle \cdot \prod_{(\pi^{\text{pre}}, \pi^{\text{post}}, B_i, B'_j) \in \mathcal{M}_{\text{max}}} \pi^{\text{pre}} \cdot \langle (g_m, g'_m) \rangle \cdot \pi_{\text{post}}^*$$

In this expression, $\prod \pi^{\text{pre}}$ denotes the *ordered concatenation* of all prefix sequences π^{pre} from the maximally aligned set $\mathcal{M}_{\text{max}}$. That is, we interpret $\prod$ as a sequence composition operator rather than a numeric product. The order of concatenation is determined by the selected branch pairs and may follow any fixed, consistent traversal.

Note that all elements in the flattened region are matched node pairs; this includes the split and merge gateways, which are explicitly represented as (g_s, g'_s) and (g_m, g'_m) , respectively.

This flattened region captures semantically consistent alignment across structurally diverging control-flow branches and is used within the EXTEND function to handle matched split gateways.

However, in some cases, a complete alignment along a single branch pair may yield a longer common subsequence than the flattened structure. To ensure that such alignments are not excluded, we define a fallback mechanism:

Definition 15 (Fallback Alignment). *Let π_{ij} be the full LCES alignment for a branch pair (B_i, B'_j) , and let $\mathcal{M}_{\text{max}}$ and π_{post}^* be as defined above.*

If

$$|\pi_{ij}| > \left(1 + \sum_{(\pi^{pre}, \pi^{post}, B_i, B'_j) \in \mathcal{M}_{\max}} |\pi^{pre}| + |\pi_{post}^*| \right),$$

then the flattened region $F(g_s, g'_s)$ is replaced by π_{ij} .

If equality holds, both variants may be retained, as either may lead to a longer overall alignment depending on subsequent continuation.

This fallback ensures that a longer, semantically meaningful alignment is not discarded merely because it bypasses structurally balanced constructs such as split and merge gateways. The fallback logic is integrated into the FLATTEN operation used within the recursive EXTEND function.

The final result is obtained by evaluating:

$$\text{LCES}(G, G') := \text{LCES}(v_{\text{start}}, v'_{\text{start}}),$$

where $v_0 \in V$ and $v'_0 \in V'$ are the unique start events of the respective BPMN models. The resulting sequence consists of matched node pairs that preserve the control-flow order and structural correspondence between the two models. This approach generalizes the classical longest common subsequence computation from linear sequences to acyclic process graphs, incorporating recursive descent, semantic node matching, and flattening of split–merge regions to align complex execution paths.

A complete pseudocode implementation of the LCES algorithm is provided in Appendix A. It mirrors the formal definitions presented here and includes recursive matching logic, memoization, branch-level alignment, flattening of parallel regions, and fallback handling. The listing is intended as a high-level, language-agnostic guide to support both conceptual understanding and practical implementation.

3.2.4 LCES Computation Example

To illustrate the behavior of the LCES algorithm, we present a concrete example involving two structurally similar but non-identical BPMN models. This example demonstrates the recursive matching of tasks, the flattening of balanced gateway regions, and the preservation of control-flow order in the resulting common execution subsequence.

Figure 3.4 shows two BPMN process models with partially overlapping task sets and differing control-flow constructs. The red-colored tasks, G in the upper model and H in the lower model, are unique to their respective processes and have no matching counterparts. As such, they are excluded during the task alignment phase, based on their name.

Following task alignment, we apply a reduction step to eliminate trivial gateway constructs. This includes gateways with a single incoming and outgoing flow, as well as directly nested splits of the same type. These transformations simplify the structure while preserving the original control-flow semantics, enhancing comparability between models.

The resulting task-aligned and normalized process graphs are shown in Figure 3.5.

After task alignment and normalization, we identify structurally corresponding split gateways in both models. In this example, each model contains a parallel split followed by a corresponding merge, forming a balanced block. These split–merge pairs delimit regions of control-flow divergence that the LCES algorithm handles through branch-level alignment and flattening.

To compare the parallel branches, we treat each outgoing path from the split gateways as a candidate for alignment. Let $\mathcal{B}(g_s) = \{B_1, B_2\}$ and $\mathcal{B}(g'_s) = \{B'_1, B'_2\}$ denote the sets of branches in the two models. For each pair (B_i, B'_j) , we compute a local longest common execution subsequence:

$$\pi_{ij} = \text{LCES}(u, u'), \quad \text{where } u \in B_i, u' \in B'_j \text{ are the entry nodes of the branches.}$$

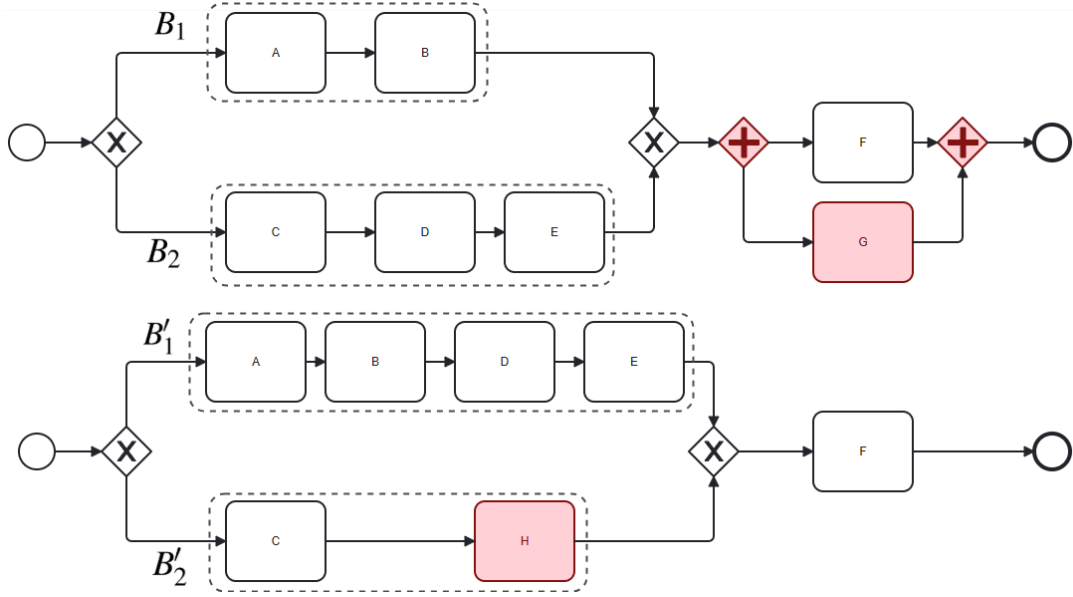


Figure 3.4: Initial BPMN models before alignment and normalization

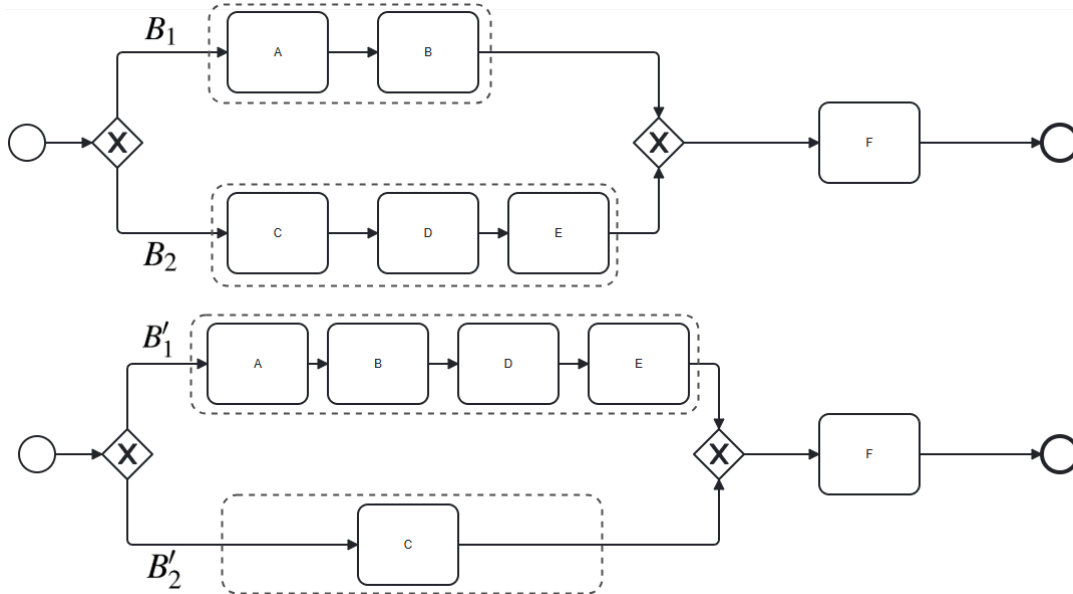


Figure 3.5: Intermediate BPMN models after task alignment and normalization

This computation proceeds recursively from the bottom up, exploring successor nodes until matching subpaths are identified. It generalizes the classical longest common subsequence (LCS) algorithm to graph-structured models with control-flow constructs such as branching and merging.

The resulting alignments are:

- $\pi_{11} = \langle (A,A), (B,B), (g_m, g'_m), (F,F), (end, end) \rangle$
- $\pi_{12} = \langle (g_m, g'_m), (F,F), (end, end) \rangle$
- $\pi_{21} = \langle (D,D), (E,E), (g_m, g'_m), (F,F), (end, end) \rangle$
- $\pi_{22} = \langle (C,C), (g_m, g'_m), (F,F), (end, end) \rangle$

Each π_{ij} represents a potential alignment between two individual branches and serves as a candidate for the flattening operation defined in the EXTEND function of the LCES algorithm. To construct the flattened region, we now decompose each alignment into a pre-merge prefix and a post-merge continuation by splitting at the matched merge gateways. The gateways themselves are excluded from the prefix.

The resulting sequences are:

- $\pi_{11}^{\text{pre}} = \langle (A,A), (B,B) \rangle, \quad \pi_{11}^{\text{post}} = \langle (F,F), (end,end) \rangle$
- $\pi_{12}^{\text{pre}} = \emptyset, \quad \pi_{12}^{\text{post}} = \langle (F,F), (end,end) \rangle$
- $\pi_{21}^{\text{pre}} = \langle (D,D), (E,E) \rangle, \quad \pi_{21}^{\text{post}} = \langle (F,F), (end,end) \rangle$
- $\pi_{22}^{\text{pre}} = \langle (C,C) \rangle, \quad \pi_{22}^{\text{post}} = \langle (F,F), (end,end) \rangle$

Among these, the pair π_{11} and π_{21} achieves the highest total pre-merge alignment length of $2 + 2 = 4$. However, this combination is *not valid* because both alignments reuse the same target branch B'_1 from the second model, violating the disjointness requirement in Definition 14.

We therefore select the valid subset with the next-highest total pre-merge length:

$$\mathcal{M}_{\text{max}} = \{(\pi_{11}^{\text{pre}}, \pi_{11}^{\text{post}}, B_1, B'_1), (\pi_{22}^{\text{pre}}, \pi_{22}^{\text{post}}, B_2, B'_2)\}$$

with a combined pre-merge length of $2 + 1 = 3$. This subset includes pairwise disjoint branches in both models and maximizes alignment under the constraints.

All valid post-merge continuations are identical, so we select:

$$\pi_{\text{post}}^* = \langle (F,F), (end,end) \rangle.$$

Putting this together, the flattened region constructed from the selected branch alignments is:

$$\begin{aligned} F(g_s, g'_s) &= \langle (g_s, g'_s) \rangle \cdot \pi_{11}^{\text{pre}} \cdot \pi_{22}^{\text{pre}} \cdot \langle (g_m, g'_m) \rangle \cdot \pi_{\text{post}}^*, \\ &= \langle (g_s, g'_s), (A,A), (B,B), (C,C), (g_m, g'_m), (F,F), (end,end) \rangle. \end{aligned}$$

This region combines aligned fragments from two structurally distinct branches, bounded by matched split and merge gateways, and followed by a shared post-merge continuation. It reflects the algorithm's ability to extract semantically coherent regions spanning parallel paths.

We now compare this result to the best full fallback alignments, which follow a single branch and may include both matched gateways. For instance:

$$\begin{aligned} \pi_{11} &= \langle (g_s, g'_s), (A,A), (B,B), (g_m, g'_m), (F,F), (end,end) \rangle, \\ \pi_{21} &= \langle (g_s, g'_s), (D,D), (E,E), (g_m, g'_m), (F,F), (end,end) \rangle. \end{aligned}$$

Each of these fallbacks contains six matched node pairs.

The flattened region, by contrast, includes three pre-merge matches $(A,A), (B,B), (C,C)$, both gateways $(g_s, g'_s), (g_m, g'_m)$, and the two-node continuation $(F,F), (end,end)$, yielding a total length of seven. Since the flattened alignment is strictly longer than any fallback alignment, the algorithm correctly selects it as the preferred result. This highlights the benefit of structural flattening: it enables the aggregation of semantically aligned fragments from separate branches into a longer, unified execution subsequence. The final common execution subsequence (CES) returned by the algorithm

starts from the matched start events and includes the flattened region described above. Combining all matched pairs in order, we obtain:

$$\Pi = \langle (start, start), (g_s, g'_s), (A, A), (B, B), (C, C), (g_m, g'_m), (F, F), (end, end) \rangle.$$

This sequence constitutes the longest semantically and structurally consistent execution path shared by the two models. It reflects not only task-level similarity, but also alignment of control-flow constructs such as parallel gateways.

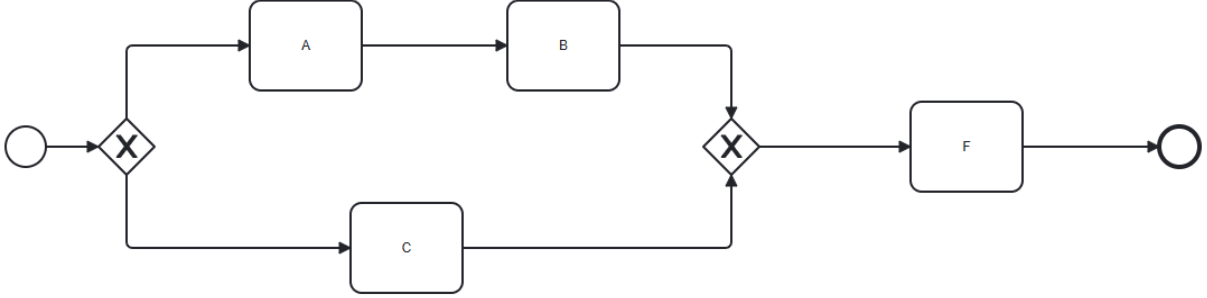


Figure 3.6: Final common execution subsequence

3.2.5 Complexity Analysis

We analyze the computational complexity of computing the longest common execution subsequence (LCES) as defined in the previous section. This analysis is grounded in the formal constructs of matched nodes, balanced split-merge regions, and precedence-preserving subsequences, and focuses on the asymptotic behavior of the recursive alignment procedure.

Given two acyclic balanced BPMN graphs $G = (V, E)$ and $G' = (V', E')$, the recursive procedure considers subproblems of the form $\text{LCES}(v, v')$, where $(v, v') \in V \times V'$. Since each node pair is evaluated at most once through memoization, the total number of distinct subproblems is bounded by $\mathcal{O}(|V| \cdot |V'|)$. For simplicity, we write $n = \max(|V|, |V'|)$, resulting in $\mathcal{O}(n^2)$ total recursive entries.

The complexity of each subproblem depends on the control-flow structure around the nodes v and v' . In particular, when both nodes are split gateways, the algorithm attempts to align their respective outgoing branches. If each split has at most d successors, there are at most d^2 branch pairs to consider. For each such pair (u, u') , the algorithm computes a local LCES, which may yield multiple distinct alignment paths of equal maximal length. Let p denote the maximum number of such alternative LCES paths retained for any branch pair. These alternatives are preserved to support semantically consistent recombination within flattened regions. The flattening step then explores combinations of these paths that are mutually compatible (i.e., they do not reuse branches). Since each of the d^2 branch pairs can contribute up to p options, the number of valid combinations is bounded by $\mathcal{O}(p^{d^2})$. This exponential term captures the combinatorial nature of aligning parallel branches across models.

After collecting candidate alignments, the algorithm selects a maximally aligned subset and evaluates the best post-merge continuation. Additionally, it compares this flattened structure against full-path alignments through individual branch pairs, adopting a fallback strategy when beneficial. These operations are polynomial in the size of the paths being compared, but they are applied to all valid combinations, making the per-subproblem cost $\mathcal{O}(p^{d^2} \cdot \ell)$, where ℓ is the maximum path length.

Multiplying the number of subproblems $\mathcal{O}(n^2)$ with the per-subproblem cost yields the total worst-case time complexity:

$$\mathcal{O}(n^2 \cdot p^{d^2} \cdot \ell)$$

The associated space complexity is $\mathcal{O}(n^2 \cdot p \cdot \ell)$ when storing all alignment paths per subproblem, or $\mathcal{O}(n^2)$ when storing only alignment lengths or a single best path per entry.

Although the exponential term p^{d^2} may appear intractable, BPMN models in practice tend to exhibit modest parallelism, with branching degrees typically being equal or lower than $d \leq 3$ or 4. Moreover, the number p of meaningful alternative alignment paths per branch pair is typically small. This is not only due to modeling discipline, but also due to structural constraints: it is rare for two process branches to admit a large number of distinct execution alignments that are all equally long and semantically valid. Even when variability exists—such as optional steps, reordered activities, or alternative routes—it usually affects only a small subset of the model and results in a limited set of viable matchings. As a consequence, the number of maximal-length LCES paths between two branches is naturally bounded in most cases, even without enforcing an explicit limit.

To improve scalability, several optimizations can be applied. If only the longest path is retained per node pair, the number of branch alignment combinations drops from p^{d^2} to 2^{d^2} , since only subsets of branch pairs are considered, rather than all permutations of alignment paths. However, this simplification comes at a cost: potentially valuable combinations of distinct paths may be missed, particularly in cases where structurally similar branches yield ties in alignment length but differ semantically. As a result, merge regions may be under-aligned, and some CES segments may be truncated prematurely.

Alternatively, limiting each node pair to at most k representative paths (e.g., top-ranked or structurally diverse options) yields a more balanced trade-off. This reduces the number of combinations to $\mathcal{O}(k^{d^2})$, resulting in overall complexity $\mathcal{O}(n^2 \cdot k^{d^2} \cdot \ell)$, where $k < p$ is a user-defined bound. This design strikes a balance between alignment quality and tractability by retaining only the most informative paths per region, thereby reducing the search space without sacrificing the ability to capture structurally significant matches.

Finally, heuristic strategies—such as greedy selection, beam search, or early pruning of inconsistent branches—can avoid explicit enumeration of all branch combinations. While these approaches forgo worst-case optimality, they offer a potential means of improving scalability and may provide acceptable alignment quality for large or deeply nested process models, though their practical effectiveness remains to be systematically assessed.

Taken together, these optimizations enable scalable computation of structurally meaningful execution alignments, while maintaining the semantic guarantees of the LCES framework. Nonetheless, they should be applied with awareness of their potential to limit alignment depth, branch coverage, or completeness in edge cases.

3.2.6 Optimality of the LCES Algorithm

The LCES algorithm is designed to compute a common execution subsequence by aligning matching tasks while respecting control-flow structure. In doing so, it relies on identifying structurally corresponding regions—particularly split–merge blocks—and aligning their branches using a combination of flattening and fallback strategies. However, this alignment procedure operates strictly within the boundaries of matched gateway pairs and does not perform any global restructuring or reordering of control-flow elements. As a result, the computed CES is guaranteed to be structurally consistent, but not necessarily optimal in length.

A key consequence of this region-local design is that the algorithm cannot reconcile structural mismatches between models that represent similar behavior using different nesting or flattening of

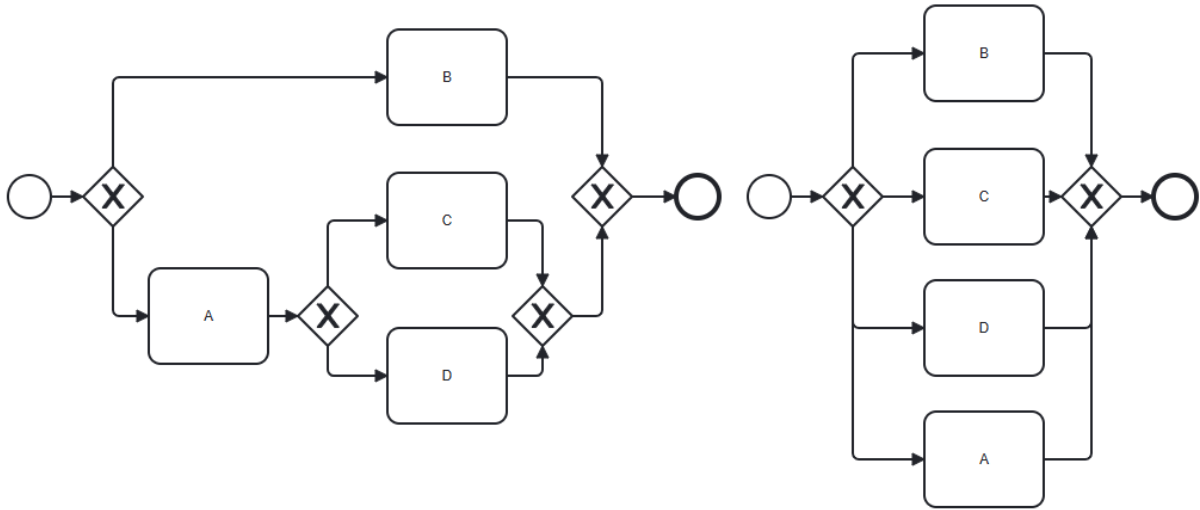


Figure 3.7: Example where a longer CES could be obtained by temporarily excluding task A, merging gateway regions, and re-inserting A afterward. The current algorithm cannot perform such restructuring.

gateways. Figure 3.7 illustrates such a case. The left model contains two nested XOR gateways: the first between tasks A and B, and the second between C and D, following the execution of A. The right model, in contrast, presents a single flattened XOR gateway over all four tasks.

Under the current strategy, the algorithm must choose between aligning the outer XOR region (e.g., A, B) or the inner one (e.g., C, D), but it cannot align both. Although the models are structurally different, a longer CES could be obtained by excluding A in both models and merging the exclusive gateways in the left model, resulting in a CES that contains B, C and D. This kind of restructuring falls outside the scope of the algorithm. Since it does not support control-flow rewriting or deferred reinsertion of matched nodes, it cannot capture such globally improved alignments. This highlights a fundamental limitation of the current gateway-bounded approach: the resulting CES is correct by construction, but may be suboptimal in capturing structurally equivalent or near-equivalent behavior when modeled differently.

3.3 Beam Search Approach

Beam search is used when the BPMN models contain loops or are structurally unbalanced—scenarios in which no reliable structural assumptions can be made. These models may include cycles and asymmetric control flows, which prevent consistent one-to-one alignment between elements in the original and modified versions. While we do not compute the exact graph edit distance (GED), our approach is inspired by the same underlying principle: modeling structural differences through sequences of edit operations. As outlined in Section 2.2.1, computing the optimal edit path between two graphs is an NP-hard problem. Exact algorithms such as A* are impractical for real-world BPMN models due to their exponential complexity. Beam search provides a tractable alternative by exploring a bounded number of promising transformation paths, enabling efficient approximation of meaningful model differences without relying on structural regularities.

Beam search provides a heuristic solution for identifying a plausible sequence of transformation steps. Rather than exhaustively exploring all possible alignments between the initial and modified

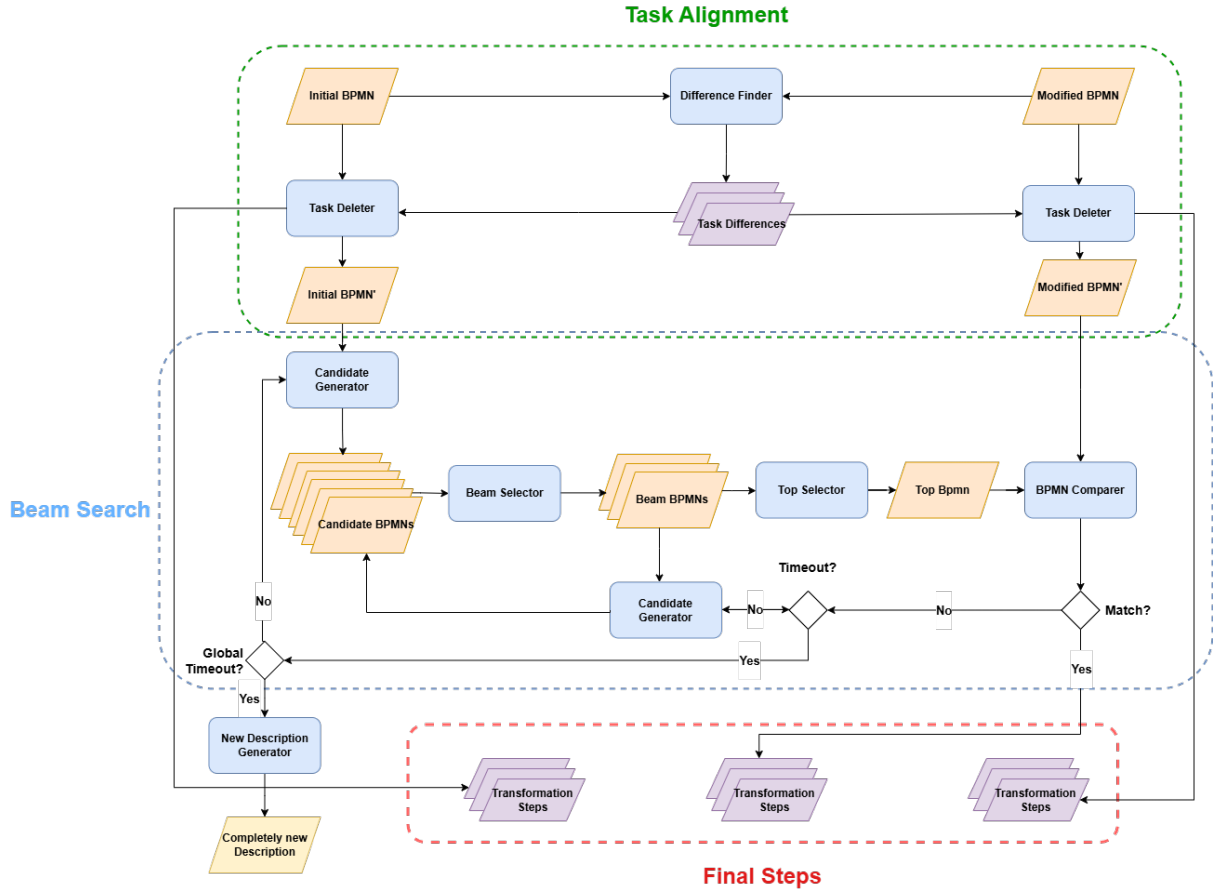


Figure 3.8: Overview of the beam search strategy

BPMN models, it maintains a fixed number of promising candidates (the “beam”) at each stage of the search. Each candidate represents a partial transformation path, extended iteratively by applying a set of domain-specific edit operations that reflect typical BPMN modifications. These include moving tasks to different positions within the control flow, adding or removing sequence flows, and inserting or deleting gateway nodes. Such operations allow the search to model a wide range of structural edits while maintaining semantic coherence in the process logic.

At every step, candidates are evaluated using a cost function that prioritizes structural alignment with the modified model while penalizing unnecessary or disruptive changes. Only the top- k candidates with the lowest estimated cost are retained and expanded in the next iteration. This strategy ensures that the search remains computationally efficient while still considering a diverse set of plausible transformation paths.

Unlike in the LCES approach, the intermediate models explored during beam search are not guaranteed to be strictly valid BPMN models at every step. In particular, the search may temporarily allow structurally relaxed constructs such as *empty gateways*—gateways with only one incoming and one outgoing sequence flow—and *inconsistent nesting* of control-flow elements, such as an exclusive gateway embedded within a parallel branch in a way that violates BPMN semantics. These deviations are tolerated during the search to improve flexibility and are corrected by the end of the transformation path. The final model is fully BPMN-compliant, and the intermediate steps remain sufficiently well-formed to support coherent text updates.

Figure 3.8 illustrates the overall beam search procedure. The process begins with the two input models: the *Initial BPMN*, representing the original process, and the *Modified BPMN*, which reflects the updated version after user-driven changes.

As a first step, a *task alignment phase* is performed to identify tasks that are present in only one of the two models. These unmatched tasks are interpreted as insertions or deletions, and are removed to produce reduced versions of the models, referred to as *Initial BPMN'* and *Modified BPMN'*. These serve as the input to the actual beam search.

The beam search is then initiated from *Initial BPMN'*. In the first iteration, all possible candidate BPMNs are generated by applying valid edit operations (e.g., moving tasks, adding or removing flows, modifying gateways). These candidates represent the immediate successors of the initial state. From this set, the top- k BPMNs are selected based on a cost function, which evaluates their proximity to *Modified BPMN'*.

The top-ranked candidate is then compared to *Modified BPMN'*. If they are structurally identical, the search terminates successfully and the corresponding sequence of operations is retained as the solution. If not, the algorithm checks whether a local timeout has been reached. If no timeout has occurred, the top- k BPMNs are used to generate the next layer of candidate BPMNs through further edit operations. The process then repeats: candidates are scored, the top- k are selected, and the best is compared to the target.

In case of a local timeout, the search strategy is adapted—either by increasing the beam width, changing the heuristic, or altering the cost distribution of edit operations—and restarted from the beginning. If a global timeout is reached, indicating that no valid transformation path can be found within reasonable computational bounds, the system defaults to generating an entirely new textual description of the modified BPMN model. This fallback ensures that a result can still be produced, even if it is no longer anchored to the original description.

The final transformation sequence consists of two parts: the initial operations inferred from task alignment (i.e., deletions and insertions based on unmatched tasks), and the structural edits discovered during beam search. Together, these steps define a complete transformation path from the original BPMN model to its modified form, which can then be used to drive the subsequent generation process.

While beam search does not guarantee an optimal or minimal sequence of changes, it also does not guarantee that a solution will be found within the given time or resource constraints. However, it provides a practical means of addressing the structural complexity often present in real-world BPMN models. By operating without strong assumptions about model regularity, beam search is capable of handling a wide range of modifications, particularly those involving loops or unbalanced control flow. This makes it a flexible and scalable strategy for process editing in settings where models frequently deviate from idealized, well-structured forms.

3.3.1 Transformation Operations

The beam search explores alternative versions of a BPMN model by applying a set of domain-specific edit operations. These operations define the transformation space and enable the construction of valid paths from the initial to the modified model. The objective is not to infer the exact changes made by a user, but rather to identify any structurally valid sequence of operations that transforms the initial BPMN into the modified one.

The supported operations include moving tasks, adding and deleting sequence flows, and inserting or removing gateways. Although only task movement is implemented explicitly, it implicitly involves removing a task from one location and re-inserting it elsewhere. This design choice is intentional: by limiting the transformation space to task movements rather than allowing unrestricted task insertions

and deletions, the search process is encouraged to preserve structural elements that are common to both the initial and modified models. This prevents the search from favoring degenerate solutions in which all tasks are removed and re-added, discarding any existing structure. Instead, movement operations promote minimal, localized modifications and help retain continuity between the original and updated process representations.

A task can be removed from its current position using one of two strategies. The primary method, which is always valid, is to bypass the task by directly connecting its preceding and succeeding nodes. This preserves continuity in the control flow and maintains graph connectivity. Alternatively, if the task lies between two gateways, it may be removed along with its adjacent sequence flow—provided that the resulting process model remains connected and all nodes are still reachable from the start event.

A task can be inserted in two structurally valid ways. First, it may be placed between two connected nodes, effectively splitting an existing sequence flow; this form of insertion is always permitted. Second, it may be inserted between a gateway that is not a merge and another gateway that is not a split. This constraint ensures that no gateway simultaneously acts as both a split and a merge, preserving valid control flow semantics and avoiding ambiguous routing constructs.

Sequence flows can be added under similar constraints. A new flow may only be inserted between a gateway that is not a merge and another that is not a split—for the same reason: to ensure that gateway roles remain clearly defined and structurally sound.

Sequence flows can be deleted under two specific conditions. First, the flow must connect two gateway nodes. Second, its removal must not compromise the reachability of the process model; that is, all nodes must remain reachable from the start event. These constraints ensure that flow deletion does not disrupt control flow or introduce disconnected components.

Gateways can be inserted along any sequence flow to introduce new control structures. Conversely, a gateway may be deleted if it has exactly one incoming and one outgoing sequence flow—in which case it acts as a structural pass-through and can be safely removed without affecting the behavior of the process.

3.3.2 Cost Model and Heuristic Function

The beam search strategy follows the classical cost estimation formulation:

$$f(n) = g(n) + h(n)$$

where $f(n)$ represents the total estimated cost of reaching the modified BPMN model from the initial model via an intermediate state n . The term $g(n)$ denotes the cumulative cost incurred so far—that is, the sum of the costs of operations applied to reach n —while $h(n)$ is a heuristic estimate of the remaining cost required to complete the transformation.

In this work, $g(n)$ is computed using a uniform cost model: each supported operation contributes a fixed, constant cost. While it is generally possible to assign distinct costs to different operation types—reflecting their structural or semantic impact—this work deliberately avoids such weighting. Empirical observations show that unbalanced cost assignments can distort the search process: for instance, if task movement is assigned a higher cost than gateway insertion, the search may favor inserting excessive gateway nodes to avoid a more appropriate task move. Conversely, if task movement is too cheap, the search may neglect gateway edits even when structurally necessary. A uniform cost model mitigates these risks and ensures that the search is guided by structural validity rather than cost-induced artifacts.

The heuristic component $h(n)$ is designed to provide a lightweight yet informative estimate of how structurally different the current BPMN candidate is from the target model. It operates by comparing control-flow structures using a canonical representation of sequence flows, called *Structural Edges*.

Structural Edge Abstraction

To compare control-flow elements robustly, each sequence flow $(u,v) \in E$ is mapped to a *Structural Edge*, defined by context-sensitive hashes of its source and target nodes. Equality between Structural Edges does not depend on node identity, but instead reflects the structural context each node occupies within the process model. In particular, this abstraction captures the local control-flow neighborhood surrounding a node—such as its adjacent tasks or gateways—rather than its absolute position or identifier.

Definition 16 (Structural Edge Equality). *Let $(u,v) \in E$ be a directed edge in a BPMN graph $G=(V,E,\lambda,\theta,v_{start},v_{end})$. Define its Structural Edge representation as:*

$$\text{STRUCTURALEDGE}(u,v) := (\text{STARTHASH}(u), \text{ENDHASH}(v))$$

where:

- For tasks:

$$\text{STARTHASH}(v) = \text{ENDHASH}(v) = \text{HASH}(\theta(v), \lambda(v))$$

- For start or end events:

$$\text{STARTHASH}(v) = \text{ENDHASH}(v) = \text{HASH}(\theta(v))$$

- For gateways:

$$\begin{aligned} \text{STARTHASH}(v) &= \text{AGGREGATE}(\theta(v), \{\text{STARTHASH}(p) \mid p \in \text{PREV}(v)\}) \\ \text{ENDHASH}(v) &= \text{AGGREGATE}(\theta(v), \{\text{ENDHASH}(s) \mid s \in \text{NEXT}(v)\}) \end{aligned}$$

Two Structural Edges (u_1, v_1) and (u_2, v_2) are considered equal if and only if:

$$\text{STARTHASH}(u_1) = \text{STARTHASH}(u_2) \quad \text{and} \quad \text{ENDHASH}(v_1) = \text{ENDHASH}(v_2)$$

The AGGREGATE function ensures structural consistency and order-independence across control-flow constructs. Given a node v and a multiset of hashes $H = \{h_1, h_2, \dots, h_n\}$ derived from adjacent nodes, the function first sorts the elements of H to enforce a canonical order. It then combines them with the hash of the node's type using a folding operation:

$$\text{AGGREGATE}(\theta(v), H) = \text{FOLD}(\text{HASH}(\theta(v)), \text{SORT}(H))$$

This ensures that structurally similar subgraphs yield identical hash values, even if the ordering of their components differs. The folding function may use a polynomial accumulator, rolling hash, or any other deterministic combiner that preserves sensitivity to structural variation.

Rather than asserting semantic equivalence, this abstraction captures localized structural similarity. In practice, it enables consistent matching of edges whose source and target nodes have identical structural context under the defined hashing scheme. This strict equality criterion ensures that only control-flow fragments with equivalent local neighborhoods are considered matches, thereby supporting stable and conservative structural alignment across model versions.

Heuristic Function.

Using Structural Edges, the heuristic function $h(n)$ quantifies the difference between the current candidate and the target BPMN model as follows. Let E_n denote the set of Structural Edges in the current candidate graph, and E_T the set in the target. The heuristic is computed as:

$$h(n) = |E_T| + 0.5 \cdot (|E_n \setminus E_T|) - |E_n \cap E_T|$$

This scoring function penalizes missing edges (i.e., those present in the target but not in the candidate) by 1.0, and extra edges (i.e., those present in the candidate but not in the target) by 0.5. Matching edges subtract 1.0 from the score, effectively rewarding structural overlap. The resulting value is always non-negative and serves as a heuristic estimate of the remaining transformation effort.

Although the heuristic is not necessarily admissible in the strict sense, this is not a requirement for beam search, which emphasizes exploration efficiency over optimality guarantees. The asymmetric weighting is deliberate: surplus edges—often introduced temporarily during the construction of new control-flow regions—are penalized less heavily to allow exploratory growth. This design also helps mitigate oscillatory behavior caused by frequently undoing and reapplying similar operations. Overall, the heuristic is computationally efficient, robust to minor syntactic variation, and well suited to guiding the search through the complex, high-branching state space of BPMN model transformations.

The Structural Edge-based heuristic is particularly well suited for transformation-guided search, as it enables fine-grained, interpretable comparisons between candidate and target models. Rather than relying on global graph similarity, it evaluates alignment at the level of individual control-flow elements, making it sensitive to the localized changes introduced during incremental edits. This granularity is essential in beam search, where candidates evolve through small structural modifications, and the heuristic must distinguish promising partial solutions from less aligned ones. The approach is also inherently directional: it evaluates progress toward a specific modified model, rather than symmetric similarity between graphs.

While alternative techniques such as the Weisfeiler-Lehman (WL) graph kernel [21] offer a principled approach to measuring global structural similarity, their application as heuristics in transformation search presents challenges. WL-based methods aggregate information over neighborhoods and lack the fine-grained resolution and directional focus required for guiding incremental transformations. Nonetheless, exploring hybrid or approximate kernel-based heuristics remains an interesting direction for future work.

3.3.3 Practical Considerations

Heuristic Limitations

The structural-edge-based heuristic is effective for guiding transformations involving small to moderately sized changes, where differences in control-flow structure can be captured through localized comparisons. However, its effectiveness diminishes when constructing larger or deeply nested regions. This limitation stems from the recursive nature of the hashing scheme: for gateway nodes, the computed hash depends on the hashes of their structural neighbors—either predecessors or successors. As a result, the heuristic fails to recognize partially constructed control-flow patterns as progress, since the hash values only converge once the full structure has been assembled.

This can lead to delayed heuristic rewards for promising transformations, particularly in scenarios involving chains of gateways, the gradual buildup of nested parallel regions, and the gradual buildup

of nested conditional regions. In such cases, the search may stall or prioritize less relevant candidates. While the current approach offers computational efficiency and robustness for smaller edit sequences distributed across the model, it remains a coarse approximation for structurally complex transformations, especially when creating entirely new control-flow constructs.

That said, the search space is partially constrained by the initial task alignment: tasks that appear in both the initial and modified BPMN models are assumed to retain their identity and are typically not relocated significantly. As a result, the beam search focuses primarily on adjusting control flow around these anchor points rather than exploring radically different task orderings. When large-scale structural changes do occur—such as the addition of new control-flow branches or nested gateways—they often introduce broader textual shifts as well, which can lead to difficulties during the prompting phase. These limitations and their impact on the text generation process are discussed further in Section 4.2.6.

Timeout Handling and Incomplete Search

Due to the combinatorial nature of the transformation space, beam search is bounded by both local and global timeouts to ensure tractability. A local timeout defines the maximum number of iterations the search is allowed to perform for a given configuration. If no matching model is found within this limit, the search is restarted with adjusted parameters—for instance, a wider beam width, an alternative cost distribution, or a modified heuristic. This allows the algorithm to escape local plateaus and explore different parts of the search space without committing excessive resources.

If a valid transformation sequence still cannot be found within the global timeout, the method falls back to an alternative strategy: instead of generating an updated description by transforming the original text, a completely new description is generated from scratch based solely on the modified BPMN model. While this approach does not preserve continuity with the original wording or structure, it guarantees that an updated description is produced even in cases where no valid transformation path can be found between the initial and modified models.

This fallback represents a pragmatic compromise between transformation fidelity and completeness, ensuring that results can be delivered even in structurally extreme cases.

Beam Width and Search Trade-offs

The beam width parameter controls the number of candidate states retained at each step of the search. A larger beam width allows the algorithm to explore a broader set of possible transformation paths, increasing the likelihood of finding a valid or more optimal sequence. However, this comes at the cost of greater computational effort and slower iteration times. Conversely, a narrower beam improves efficiency but risks pruning promising candidates too early. In this work, the beam width is selected empirically based on a balance between runtime constraints and search quality. While dynamic adjustment strategies are not implemented, the architecture allows for adaptive tuning in future extensions.

Together, these elements define a robust yet flexible framework for identifying structural transformations in complex BPMN models, supporting the overarching goal of minimal and context-aware text revision.

3.4 GPT-Based Prompting Strategy

After computing a transformation path between the initial and modified BPMN models—using either the LCES approach or the beam search heuristic—the final step is to update the corresponding process

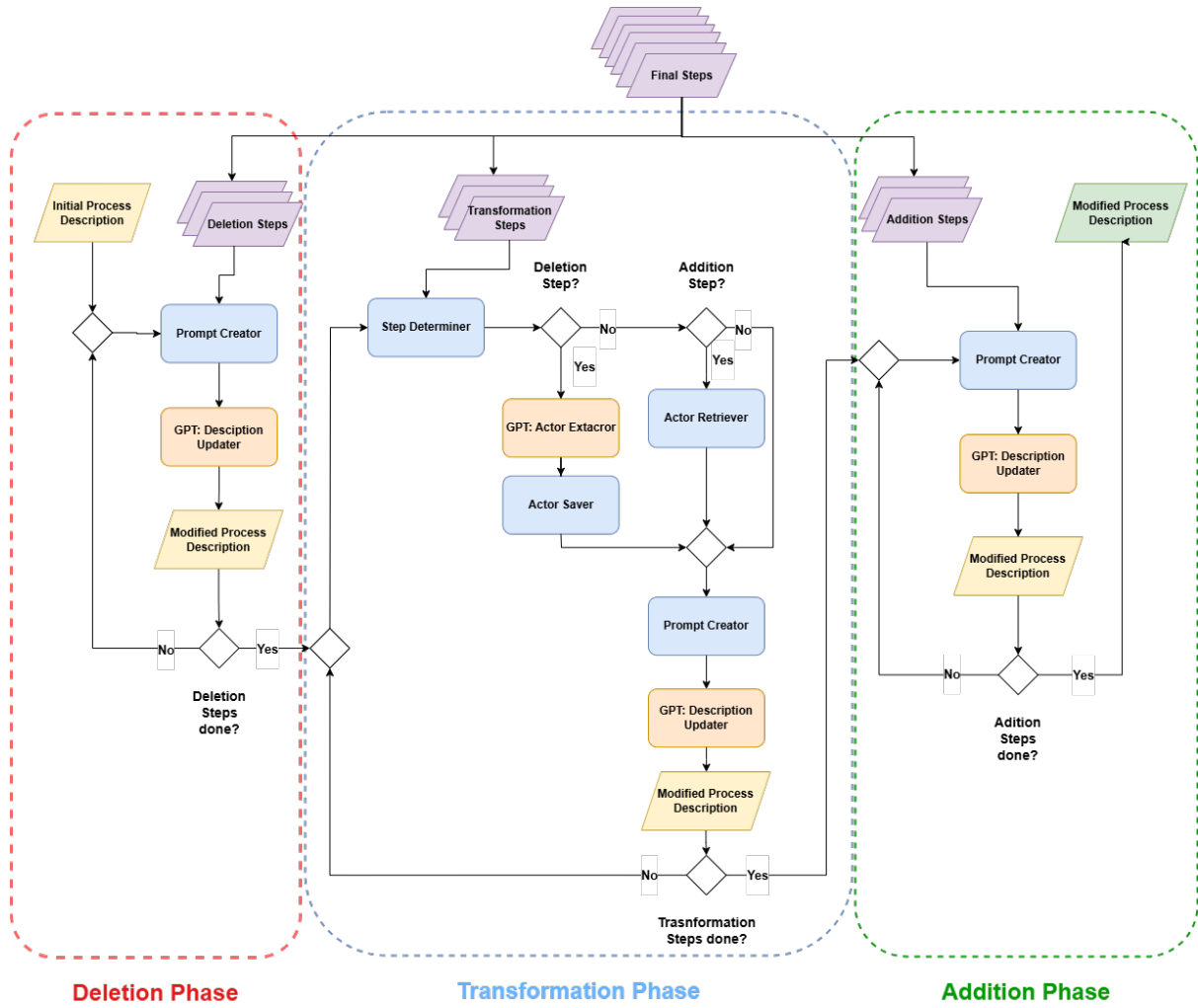


Figure 3.9: Three-phase strategy for integrating BPMN modifications into the process description using GPT

description. Rather than generating the entire text anew, the focus is on applying targeted, incremental edits that preserve the original wording and structure as much as possible.

To achieve this, each transformation step is converted into a natural-language instruction and passed to GPT. The model is guided to revise the process description accordingly, while keeping all unrelated content unchanged. This strategy ensures that updates remain localized, coherent, and stylistically consistent.

As illustrated in Figure 3.9, the prompting process is divided into three sequential phases: deletion, transformation, and addition. Each phase corresponds to a distinct category of transformation steps and is applied in order.

The **deletion phase** and **addition phase** handle steps generated during the task alignment stage of the LCES or beam search workflows. These always involve tasks that appear in only one of the two models and therefore must be removed or inserted. For each deletion, a prompt is formulated to eliminate the corresponding sentence or phrase from the current description. Conversely, additions involve inserting a new task description into the appropriate position, based on structural context.

The **transformation phase** applies the core sequence of structural changes computed by the LCES or beam search algorithms. In the case of LCES, these include task additions and deletions

within shared control structures, as well as changes to gateway types. For beam search, transformation steps include task movements—internally modeled as a deletion followed by an addition—as well as the insertion and removal of sequence flows between gateways. While the underlying model may also involve the insertion or removal of gateways, such modifications are not prompted explicitly. Instead, they are handled implicitly through the context of task-level changes: when a task is added or removed, the surrounding control-flow constructs adjust accordingly based on the structural dependencies provided to GPT.

When a task is deleted and later reintroduced during the transformation phase, the actor responsible for the task—such as an “employee”, “system”, or specific organizational role—is first extracted from the original description using GPT. This extraction occurs at the time of deletion, provided that the actor is explicitly mentioned in the text. The extracted information is then reused when re-adding the task, allowing the prompt to maintain narrative consistency and preserve role continuity in the resulting description.

Across all three phases, prompts are constructed with precise instructions, including the name and position of the task being changed and relevant contextual anchors. Each prompt targets a localized portion of the description and includes structural cues—for instance, the tasks that come before or after the change—to help GPT integrate the edit in the correct position. GPT then applies the update incrementally, preserving the surrounding text. This strategy ensures that the final description accurately reflects the updated process model while retaining the original tone, structure, and vocabulary.

3.4.1 Prompting Methodology

To ensure that each BPMN model transformation is reflected accurately in the corresponding process description, we follow a consistent strategy for constructing GPT prompts. This section outlines the core design principles that guide all prompt generation in our approach. These principles—focused on maintaining structural fidelity, narrative consistency, and localized edits—are applied systematically across different transformation types, as detailed in the following subsections.

Each structural change—such as task insertion, removal, or control-flow modification—is treated as a localized transformation. Rather than regenerating the full text, our approach prompts GPT to revise only the affected parts of the description, preserving the original structure, tone, and vocabulary.

Each prompt is composed of several components: a natural-language instruction describing the intended transformation, the complete original process description, a set of localized dependency statements reflecting the control-flow context, and a set of constraints that guide GPT’s behavior. These elements work together to ensure that each transformation is applied precisely, consistently, and with minimal disruption to the surrounding content.

Full-Context Editing

Each prompt includes the complete original process description. This ensures that GPT has access to the full context of the workflow—not only its content, but also its tone, phrasing, and narrative structure. Since the objective is to make minimal, localized edits, the model must align stylistically with the existing text while preserving its structure.

Including the full description also avoids the need to manually isolate a fragment. This is especially important for changes involving long-range dependencies, such as loops or shared control-flow structures, where a task may be linked to sentences spread across different parts of the text. Providing the full narrative allows GPT to integrate changes naturally and reduces the risk of inconsistencies or misplaced edits.

Natural-Language Dependency Encoding

Each prompt includes a minimal set of dependency statements that describe the control-flow relationships between tasks before and after the transformation. These statements help GPT determine where to apply the edit by indicating which tasks come immediately before or after the change, and whether they occur sequentially, in parallel, or under exclusive conditions.

To keep the prompt focused, we include only the local structure relevant to the change, rather than the full set of process dependencies. This helps reduce cognitive overload for the model and avoids introducing irrelevant information that could distract from the intended modification. It also improves interpretability for human reviewers and ensures the transformation remains narrowly scoped.

Dependencies are expressed using structured natural language, not a formal grammar or domain-specific syntax. Early experiments showed that GPT often misinterpreted custom grammars in a zero-shot setting. Even when examples were provided, the model tended to overfit to the phrasing, generating stylistically inconsistent text. In contrast, plain natural language yields more reliable generalization while maintaining narrative fluency.

To extract these dependencies, we perform a localized traversal over the BPMN graph. In doing so, we ignore gateway nodes that have exactly one incoming and one outgoing edge—an artifact of beam search and intermediate model states—which contribute no meaningful control-flow logic. These are skipped transparently during traversal, with no impact on the generated dependency statements.

We also omit prompt construction for transformations that involve only the insertion or removal of such gateways. Instead, the description is updated implicitly during subsequent task-level edits that introduce or remove tasks associated with those gateways. This avoids confusing GPT with intermediate structural steps that do not correspond to changes in the actual process content.

These statements are essential not only for structural alignment but also to prevent misinterpretation. Without explicit guidance, GPT may wrongly infer that unrelated tasks—such as later steps in the process—are part of the immediate control flow. Explicitly listing only the relevant local structure helps keep the edit confined to the intended region.

Behavioral Constraints and Role Framing.

To prevent unintended edits and ensure alignment with the process model, the prompt explicitly defines GPT’s role and behavioral boundaries. The model is instructed to act as a BPMN expert executing a specific transformation, without inferring missing steps, correcting errors, or modifying anything beyond what is explicitly described.

This role framing is essential because GPT is optimized for plausibility: it tends to generate descriptions that are realistic or coherent, even if they conflict with the given structure. This is particularly problematic when the BPMN model is in an intermediate state—for example, after an insertion but before other changes have been applied—which may not yet form a plausible process.

Such behavior was observed in early experiments, where GPT attempted to reorder steps or smooth over inconsistencies to improve narrative quality. While fluent, these outputs often diverged from the intended control flow. The behavioral constraints reduce such drift by emphasizing obedience over realism, instructing GPT to apply only the specified change and leave the rest of the description untouched.

Localized Instruction Format.

Each prompt begins with a clear and direct instruction that specifies the type of transformation to be performed—such as inserting or removing a task, or changing the control flow. This instruction

is phrased in plain language and includes the name of the task to be added or removed, reducing ambiguity and helping GPT focus on a single, well-scoped change.

By stating the transformation type explicitly, the prompt avoids triggering a general rewriting behavior. Combined with the structural context, this format supports precise, localized edits and discourages modifications to unrelated parts of the text.

Stylistic and Semantic Constraints.

The prompt ends with a set of constraints designed to preserve the tone, structure, and semantics of the original text. GPT is instructed to maintain the narrative voice, use concise and consistent phrasing, and express control-flow constructs such as parallelism or exclusivity explicitly.

These constraints help ensure that the generated output is both faithful to the process structure and fluent in style. Together with the structural and behavioral guidance, they form a prompting strategy that is robust across transformation types, minimizes post-editing effort, and maintains semantic alignment with the BPMN model.

Assumptions and Scope.

While the process models may contain cycles or loops, these are not treated as a special case in the prompting framework. The strategy does not currently model loop semantics explicitly in the textual description; instead, loops are handled in the same way as other control-flow structures, using local dependency information. Accurately capturing the temporal or iterative logic of such constructs would require additional narrative mechanisms beyond the current formulation.

Likewise, the prompt does not include explicit anchoring instructions such as “insert after sentence X”. GPT determines the insertion or modification point implicitly, based on the control-flow dependencies provided. This avoids brittle anchoring while ensuring structural consistency in the output.

3.4.2 Task-Level Prompting

To demonstrate how the prompting principles from the previous section are applied in practice, we begin with task-level transformations involving the insertion and deletion of individual activities. These operations follow the general prompting framework, with only minor variations in the instruction and dependency encoding. The approach is illustrated using an example from a university enrollment process.

Context and Setup

The example is drawn from a university’s student enrollment process. Initially, the process begins with a review of the student’s application, followed by two parallel branches—one for academic validation and another for administrative checks. The corresponding BPMN structure is shown in Figure 3.10.

The original process description, included in full in the prompt, reads:

The process begins when the university reviews a student’s application. The workflow then proceeds in parallel along two branches. In one branch, the student’s academic qualifications are checked and their language proficiency is verified. In the other branch, the tuition payment is validated and the student’s residency status is confirmed.

We now consider the insertion of a new task, *Assign Case Officer*, immediately following the review. The resulting BPMN update is shown in Figure 3.11.

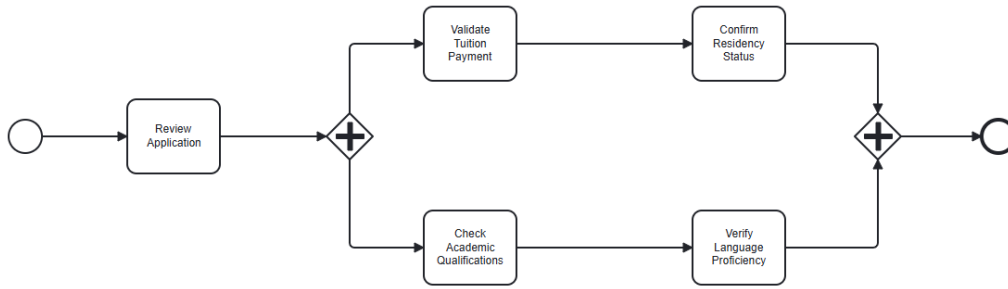


Figure 3.10: Initial BPMN structure

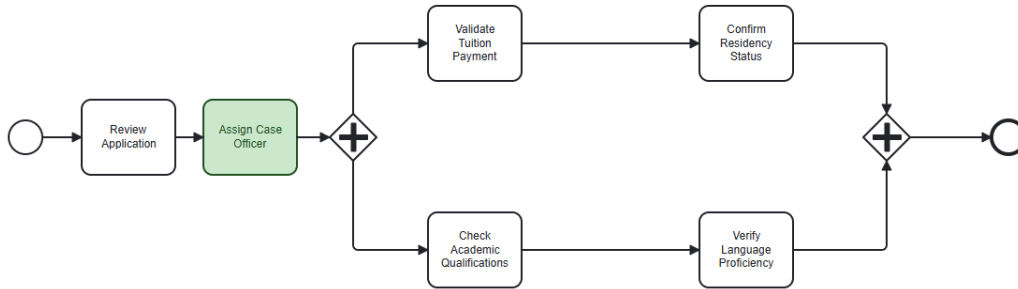


Figure 3.11: Modified BPMN structure after inserting Assign Case Officer

Instruction Format

The prompt begins with a brief instruction that clearly specifies the transformation in natural language. The name of the task to be inserted is quoted to reduce ambiguity:

Your task is to insert the task ‘Assign Case Officer’ in the process description.

Dependency Encoding

To localize the change within the process structure, the prompt provides two sets of natural-language dependency statements—one describing the control flow before the transformation and one describing it after.

These statements are generated through a localized graph traversal centered on the inserted or deleted task. We begin by identifying its immediate predecessor (parent node) and successor (child node). If the parent node is a task, it is included directly; if it is a gateway, we perform a backward traversal to collect all structurally reachable predecessors.

For the child node, we always perform a backward traversal—regardless of whether it is a task or a gateway—to identify its structurally reachable predecessors. The union of all tasks obtained from these backward traversals forms the base set from which we then perform forward searches to determine their immediate successors. This forward information reflects the local post-conditions and helps articulate how the change affects downstream execution.

The resulting structure is expressed in natural language using conjunctive phrasing for parallelism (e.g., “A and B”) and disjunctive phrasing for exclusivity (e.g., “one of A or B”). This provides a concise but precise description of the affected process region.

For the current case, inserting Assign Case Officer yields the following dependency context:

The previous dependencies in the process are: Review Application is only before (Check Academic Qualifications AND Validate Tuition Payment) ### The new dependencies in the process are: Review Application is only before Assign Case Officer; Assign Case Officer is only before (Check Academic Qualifications AND Validate Tuition Payment)

Stylistic and Semantic Constraints

The prompt also includes constraints on tone, scope, and fluency. GPT is instructed to match the style of the original process description, make only the requested edit, and return a complete updated version. The relevant instruction is phrased as:

Describe the task in the style of the given process description. Be as precise and short as possible. Do not change other meanings. Answer with the full description.

This ensures that the output is both stylistically aligned and structurally faithful, avoiding paraphrasing or unrequested alterations to the surrounding text.

Prompt and Output

The complete prompt includes the transformation instruction, full original description, localized dependencies, and behavioral constraints. GPT returns the following:

The process begins when the university reviews a student's application. The next step is to assign a case officer. The workflow then proceeds in parallel along two branches. In one branch, the student's academic qualifications are checked and their language proficiency is verified. In the other branch, the tuition payment is validated and the student's residency status is confirmed.

Task Deletion

Task deletion follows the same format. The instruction specifies that a task should be removed, and the dependency statements are adjusted accordingly to reflect the updated structure. The rest of the prompt—the process description, behavior constraints, and expected output format—remains unchanged. This symmetry allows task insertion and deletion to be handled using a single unified prompting template.

In cases where a task is deleted and later reinserted, additional care is taken to preserve role-related information. Before deletion, GPT is asked to identify the actor responsible for the task. This information is stored and later reused when re-adding the task. This two-step handling ensures that actor assignments are not lost during intermediate transformations and can be reintegrated into the narrative accurately.

3.4.3 Sequence Flow Prompting

In addition to task-level changes, structural modifications may involve adjustments to the control flow between existing tasks—such as the addition or removal of sequence flows. These transformations affect how tasks are ordered or grouped for execution and must be precisely reflected in the textual description. In what follows, we demonstrate how the prompting principles introduced earlier are applied to these control-flow transformations through an example.

Overview and Motivation.

We illustrate two representative cases of sequence flow addition. The first introduces a dependency between tasks in previously parallel branches, requiring synchronization before continuation. The second connects tasks across exclusive branches, expanding the set of possible execution paths. In both scenarios, the BPMN structure changes, but the task content remains unchanged.

Figures 3.12 and 3.13 depict these structural updates:

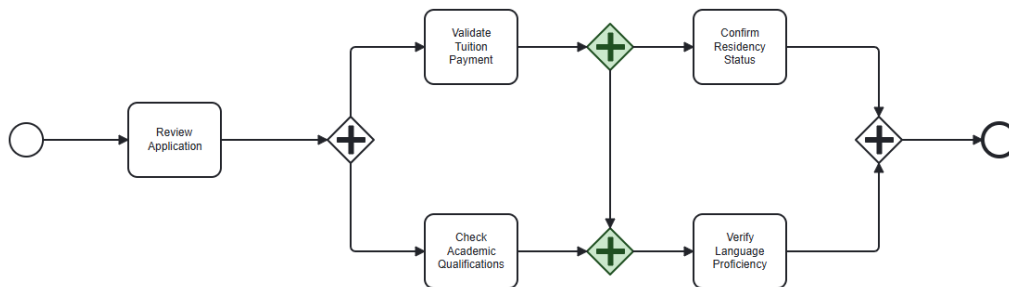


Figure 3.12: Flow addition between parallel branches: Validate Tuition Payment → Verify Language Proficiency

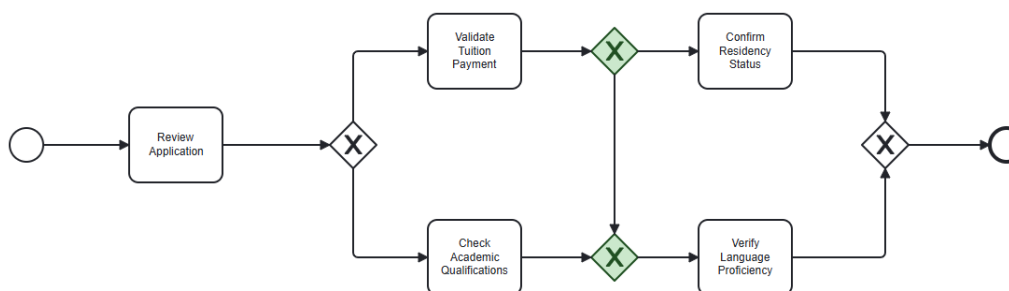


Figure 3.13: Flow addition across exclusive branches: Validate Tuition Payment → Verify Language Proficiency

Structural Interpretation and Dependency Encoding.

To express the structural change, we encode how the new sequence flow modifies task relationships using concise, structured natural language. The analysis is based on a localized traversal of the process graph surrounding the added or removed connection.

In the parallel case, the new dependency introduces a synchronization constraint: both tasks must now complete before the process proceeds. We analyze the merging gateway by tracing backward to identify incoming tasks and forward to identify successors affected by the new joint prerequisite. The semantics of parallelism are preserved: the prompt communicates that more must be completed before continuation.

In the exclusive case, the sequence flow introduces a new conditional path. Here, we examine the divergence point (split gateway). A forward search reveals new reachable tasks, while a backward search identifies new dependencies. This is phrased in terms of newly enabled alternatives, rather than strict ordering.

To avoid redundant or unnatural output, we include only a single directional perspective per change—either from the merge gateway (in parallel cases) or the split gateway (in exclusive ones).

Including both perspectives often leads GPT to repeat the same dependency information multiple times in the generated description, producing verbose or stylistically inconsistent text. By focusing on just one direction, the prompt remains concise and natural, while still conveying the necessary structural change. This asymmetry also reflects the underlying transformation logic: parallel updates introduce additional synchronization, while exclusive updates expand the set of possible outcomes.

Prompt Construction and Examples.

The prompt structure follows the established framework: a brief natural-language instruction, the full original process description, a revised dependency statement, and behavioral and stylistic constraints.

The added dependency for the parallel case is phrased as:

```
Verify Language Proficiency is now after (Validate Tuition Payment and Check Academic Qualifications) finished instead of being only after Check Academic Qualifications finished.
```

For the exclusive case:

```
Validate Tuition Payment is now followed by (one of Verify Language Proficiency OR Confirm Residency Status) instead of being only followed by Confirm Residency Status.
```

Flow deletions are expressed by reversing the dependency logic:

```
Validate Tuition Payment is now only followed by Confirm Residency Status instead of being followed by (one of Verify Language Proficiency OR Confirm Residency Status).
```

Stylistic and Behavioral Constraints.

The prompt concludes with a directive instructing GPT to preserve the style and semantics of the original description. The model is told to:

```
Return a description in natural language in the same style of the given process description. Do not change other meanings in the process description. Answer only with the full description.
```

This helps ensure that the resulting text integrates the transformation fluently while leaving unrelated content untouched. The same behavioral constraints used in earlier sections apply: GPT is not permitted to infer, generalize, or repair the structure beyond what the prompt specifies.

3.4.4 Gateway Prompting

In the balanced setting, we also consider transformations that change the type of a gateway-replacing parallel semantics with exclusive semantics, or vice versa. This modification does not alter the number or identity of the branches, nor the tasks they contain. Instead, it changes only how those branches are interpreted: whether they represent concurrent execution or a mutually exclusive choice.

To update the textual description accordingly, we identify all tasks that follow the gateway merge and express their new relationship in natural language. This updated dependency context, along with

the original process description and standard behavioral constraints, is provided to guide the revision. The change is framed as a localized transformation that preserves all other content, style, and structure. GPT is expected to reflect the updated control-flow semantics—e.g., changing a parallel phrasing to an exclusive one—without modifying unrelated parts of the text.

Tool and Experiments

This chapter presents the core implementation of the transformation system, followed by an empirical evaluation of its performance and reliability. The first part outlines the tool’s architecture, key components, and integration strategy, while the second part discusses a series of controlled experiments conducted on a variety of BPMN models. Together, these sections demonstrate both the design rationale and the practical effectiveness of the proposed approach.

4.1 Tool

4.1.1 Overview

To validate the approach described in this report, a prototype tool was developed that implements the full transformation pipeline from BPMN model comparison to textual description update. Given two BPMN models—an initial version and a modified version—together with a textual description of the initial model, the tool generates an updated version of the description that reflects the structural changes present in the modified BPMN. The objective is to produce a revised description that stays as close as possible to the original in wording and structure, while maintaining semantic accuracy.

4.1.2 System Design

The system directly implements the concepts introduced in Chapter 3. As illustrated in Figure 3.2, the tool first parses the input BPMN models, checks their structural properties, and selects either the LCES algorithm or a beam search strategy depending on whether the models are balanced and acyclic. Once a transformation path is computed, each step is used to construct a natural-language prompt, which is submitted to GPT-4.1. The model incrementally updates the description in response to these prompts, preserving phrasing wherever possible and applying edits only where structurally required.

4.1.3 Implementation Notes

The core transformation tool is implemented in Java and consists of approximately 7000 lines of code. It is structured as a command-line application. The two BPMN models are provided as `.bpmn` files in BPMN 2.0 XML format, and the initial process description is supplied as a plain text file. The resulting modified description is returned as a separate text file. Prompts are constructed dynamically

for each transformation step, incorporating the full original description along with localized structural instructions.

The tool accesses GPT-4.1 via the OpenAI API and manages prompt submission sequentially, ensuring that the output of each step is used as the input for the next. The prompting strategy follows the method detailed in Section 3.4, with careful attention to preserving the original phrasing unless structural changes demand modification.

While the Java tool functions independently as a command-line utility, it has also been integrated into an existing web-based platform for BPMN generation from textual requirements. This platform, originally introduced by Nivon and Salaün [19], allows users to input informal natural-language descriptions and automatically generates BPMN models that satisfy them¹. The transformation component developed in this work extends the platform by enabling users to manually adapt the textual description when the BPMN model is changed.

The frontend is implemented using standard web technologies including HTML, CSS, and JavaScript, enhanced by jQuery, Ajax, and Bootstrap for responsive and interactive behavior. BPMN diagrams are rendered and edited using the `bpmn.io` toolkit², which supports direct manipulation of BPMN 2.0 elements in the browser.

Users can manually trigger the description update process via a dedicated button in the interface. Upon activation, the updated BPMN model and original description are sent to a Node.js backend, which invokes the Java transformation engine. The transformation process is identical to the command-line version, and the updated description is then returned to the frontend for display.

Screenshots illustrating the interface before and after the transformation process can be found in Appendix B

4.1.4 Scope and Limitations

The current version supports BPMN models that include parallel and exclusive gateways, a single start event, a single end event, and arbitrary task labels. It assumes that all input BPMN files are syntactically valid and behaviorally sound. Additionally, the tool requires that task names are unique within each BPMN model, as task identity is determined based on name matching during transformation alignment. Advanced BPMN constructs such as subprocesses, multi-instance loops, and event-based gateways are not supported. The tool is intended for research and experimentation, and while it performs reliably under controlled conditions, further adaptation would be needed for use in production environments.

4.2 Experiments

4.2.1 Experimental Setup

The evaluation was conducted on six BPMN models, mostly based on process examples from the literature, including Friedrich [9] and the PET dataset [4]. The models were chosen to represent a range of structural and size-related characteristics. Specifically, three balanced and three unbalanced processes were selected, with one small, one medium, and one large model in each group. The number of tasks per model ranges from 5 to 16. This setup ensures that the approach is tested on BPMNs of varying complexity and structure.

¹The platform is accessible at: <https://lig-givup.imag.fr/>

²<https://bpmn.io>

For each base model, several modified variants were created to simulate typical process evolution scenarios, including additions and deletions of tasks, flows, and gateways, as well as replacements and reorderings. While replacements and reorderings are conceptually distinct operations, they are internally modeled by the tool as combinations of deletions and additions. Each modified BPMN, together with the original process description, was processed by the tool to generate an updated textual description. The transformations were kept intentionally compact to reflect realistic update scenarios: additions, deletions, and movements typically involved two to three tasks depending on model size, while loop and flow operations were limited to a single structural edit. Combined changes included one instance of each transformation type, serving as a representative compound case.

4.2.2 Manual evaluation criteria

The quality of the generated process descriptions was assessed manually along two key dimensions: *semantic correctness* and *textual style*. Each output received a score from 1 to 3 on both scales, based on the criteria summarized in Table 4.1.

Score	Semantic Score Description	Style Score Description
3	Fully correct; the process logic is accurately described.	Minimal change; style and tone match the original closely.
2	Mostly correct; plausible structure but some ambiguous or unexpected dependencies.	Somewhat intrusive or awkward phrasing.
1	Incorrect; the description fails to reflect the BPMN structure.	Significant stylistic or narrative divergence.

Table 4.1: Scoring criteria for semantic accuracy and style quality.

When a transformation involved multiple edits—such as the insertion of several tasks—each was scored individually, and the final score was computed as their average.

Manual evaluation was necessary due to the limitations of existing automatic metrics. Semantic correctness often hinges on subtle dependencies in control flow and requires domain-specific understanding of BPMN semantics. Likewise, textual style depends on tone, fluency, and narrative consistency—dimensions that common metrics such as BLEU [20], ROUGE [16], and BERTScore [25] fail to capture. These methods focus on surface-level similarity or embedding-based matching and cannot assess whether edits were minimal, appropriate, or structurally motivated.

4.2.3 Structural Alignment Metric

While automated metrics such as BLEU, ROUGE, and BERTScore are commonly used for evaluating text similarity, they are ill-suited for assessing the structural appropriateness of edits in process descriptions. To address this gap, we introduce the **Structural Alignment Score (SAS)**, a custom metric specifically designed to evaluate the alignment between a machine-generated revision and a human-edited reference. SAS does not aim to capture stylistic fluency or semantic nuance, but instead provides a focused and interpretable signal of structural faithfulness.

The goal of SAS is to assess how well a model performs minimal, semantically aligned edits in response to structural changes in a BPMN process model—preserving validated portions of the original while removing or replacing only what is necessary.

A good revision, according to SAS, should avoid unnecessary rephrasing and retain as much of the original narrative as possible. To capture this behavior, the metric rewards content preservation through a *preservation ratio* that compares retained words in the machine output against those preserved by a human editor. However, preservation alone is insufficient: a model that never deletes anything could achieve a high preservation score without making appropriate structural edits. SAS therefore introduces a second component, the *deletion F1 score*, which evaluates how precisely and completely the model replicates human deletions. Together, these two components provide a balanced view of conservative, faithful editing behavior.

We do not explicitly evaluate additions, as they are typically semantically open-ended and context-dependent. Unlike deletions—which can be judged relative to known original content—additions may vary greatly in wording while still conveying equivalent meaning. Measuring their quality through simple token overlap would be both unreliable and overly sensitive to phrasing differences. Instead, SAS indirectly penalizes irrelevant or excessive additions via the preservation ratio, since such changes tend to displace validated original content.

4.2.4 Metric Definition and Computation

The Structural Alignment Score (SAS) evaluates how closely a machine-generated revision aligns with a human-edited reference, given a shared original process description. To compute this score, we compare three versions of the text: the original input, the machine-generated output, and the human-modified reference. Each text is preprocessed by lowercasing all characters, removing punctuation, and splitting on whitespace to extract word-level tokens. Word order is disregarded throughout; the resulting representations are treated as bag-of-words (BoW) multisets that retain token frequencies but not sequence. This simplification focuses the metric on structural overlap and editing behavior, without relying on syntactic or semantic parsing.

To identify structural changes, we use Python’s `difflib.SequenceMatcher`³, which aligns the original and revised texts based on minimal edit operations. This produces a sequence of *opcodes*, each describing a contiguous span of equal, deleted, inserted, or replaced word segments. These operations are structurally interpreted as follows: spans marked as `equal` denote content preserved in both versions and contribute to the *structural intersection*; `delete` and `replace` represent removals and are considered *structural deletions*; `insert` and `replace` signal added or reformulated content and count as *structural additions*. These mappings are summarized in Table 4.2.

From these opcodes, we extract three core multisets for each comparison: I_m and I_h , which contain the words preserved from the original by the machine and the human, respectively; D_m and D_h , representing the words deleted or replaced from the original by each editor; and A_m and A_h , capturing the words newly added or substituted in the revised texts.

These multisets retain word frequencies but not positions, which is essential under the BoW assumption. They form the basis for computing a structural comparison metric that evaluates both preservation and editing fidelity in a unified and interpretable manner.

Definition 17 (Preservation Ratio).

$$R_P = \frac{|I_m \cap I_h|}{|I_h|}$$

The preservation ratio R_P quantifies how many of the tokens preserved by the human editor are also preserved by the machine. The operation $\cap$ denotes multiset intersection, capturing both token

³<https://docs.python.org/3/library/difflib.html>

Opcode Type	Meaning	Structural Set
equal	A contiguous sequence of words appearing in both texts.	Structural Intersection
delete	Words present in the original but absent in the modified version.	Structural Deletion
insert	Words introduced in the modified version but not present in the original.	Structural Addition
replace	Words in the original that are substituted by different words.	Deletion and Addition

Table 4.2: Structural interpretation of difflib opcodes under BoW assumptions

identity and frequency. A score of 1 indicates perfect agreement in preserved content; lower values indicate that the machine removed tokens the human considered valid.

Definition 18 (Deletion F1 Score).

$$\begin{aligned}
 Precision_D &= \frac{|D_m \cap D_h|}{|D_m|}, \quad Recall_D = \frac{|D_m \cap D_h|}{|D_h|} \\
 F_D &= \begin{cases} 1, & \text{if } |D_m| = 0 \text{ and } |D_h| = 0 \\ 0, & \text{if } |D_m| = 0 \text{ or } |D_h| = 0 \\ \frac{2 \times Precision_D \times Recall_D}{Precision_D + Recall_D}, & \text{otherwise} \end{cases}
 \end{aligned}$$

The deletion F1 score F_D balances precision and recall in reproducing human deletions. High precision indicates that most of the machine’s deletions were also made by the human; high recall indicates that the machine successfully covered the human’s deletions. The harmonic mean penalizes both over- and under-deletion.

Definition 19 (Structural Change Factor).

$$Factor = \frac{|A_h|}{|A_h| + |D_h|}$$

The structural change factor reflects the nature of human edits, weighting the metric toward preservation or deletion based on the relative frequency of human additions and deletions. When the human primarily adds, the metric emphasizes retention; when primarily deleting, it emphasizes deletion alignment.

Definition 20 (Structural Alignment Score (SAS)). *Let R_P denote the preservation ratio, F_D the deletion F1 score, and $Factor$ the structural change factor. The Structural Alignment Score is defined as:*

$$SAS = Factor \times R_P + (1 - Factor) \times F_D$$

This composite score lies in the range $[0, 1]$, with higher values indicating stronger alignment with human editing behavior across both preservation and deletion. It ensures that models are rewarded not only for retaining validated content but also for performing targeted, human-like deletions. By incorporating an F1-based deletion score, the metric avoids favoring either under- or over-editing, making it suitable for evaluating minimal, semantically faithful revisions.

The limitations of this metric, including sensitivity to text length and surface-level overlap, are discussed further in Section 4.2.6.

4.2.5 Results

Semantic Score

Transformation Type	SmallU	MediumU	BigU	SmallB	MediumB	BigB
Task Addition	3.0	3.0	2.7	3.0	3.0	3.0
Task Deletion	2.0	3.0	3.0	2.0	2.5	3.0
Task Replacement	3.0	3.0	3.0	3.0	1.0	3.0
Task Movement	2.5	3.0	3.0	3.0	3.0	3.0
Loop Addition	3.0	3.0	1.0	–	–	–
Flow Addition	–	3.0	3.0	–	–	–
Flow Deletion	–	3.0	3.0	–	–	–
Combined Changes	3.0	2.8	2.6	3.0	3.0	3.0

Table 4.3: Semantic evaluation scores by transformation type across all BPMN models (rounded to one decimal place).

Table 4.3 reports the semantic correctness scores for different transformation types, each applied once to a corresponding BPMN model. The models are categorized based on two structural dimensions: balance (balanced vs. unbalanced) and size (small, medium, large). The labels *SmallU*, *MediumU*, and *BigU* refer to small, medium, and large unbalanced models, respectively, while *SmallB*, *MediumB*, and *BigB* refer to their balanced counterparts. The values represent single test runs per transformation and model, rather than averages. Scores range from 1 (incorrect) to 3 (fully correct), as described in Table 4.1. A dash (–) indicates that the corresponding transformation type was not evaluated on that model, typically because the operation was either structurally inapplicable (e.g., loop insertion in a balanced model) or irrelevant for the chosen scenario.

The results show that the approach performs reliably across a range of transformation types and process structures. Task-level operations such as additions, deletions, replacements, and movements generally yielded high semantic scores, with most values at or near 3.0. Task additions, in particular, performed slightly more consistently than deletions, suggesting that introducing new content is less error-prone than removing existing elements without disrupting the surrounding flow. Loop-related transformations performed well in small and medium unbalanced models but failed in the large unbalanced model. This drop cannot be attributed solely to the size of the model; it may also reflect the absence of explicit loop-handling logic in the transformation process. Combined changes yielded high correctness in all balanced models and most unbalanced ones, suggesting that the method handles multiple simultaneous edits effectively. While structurally balanced, acyclic models allow for more predictable transformation sequences via the LCES approach, the beam search strategy also proved effective in less regular settings, though with slightly more variability. Overall, balanced and unbalanced models showed similar performance trends, and the size of the BPMN model did not appear to have a strong influence on semantic correctness. This indicates that the method generalizes robustly across different structural configurations and levels of process complexity within the tested range.

Transformation Type	SmallU	MediumU	BigU	SmallB	MediumB	BigB
Task Addition	3.0	3.0	3.0	2.5	3.0	2.0
Task Deletion	3.0	3.0	3.0	3.0	3.0	3.0
Task Replacement	3.0	2.0	3.0	3.0	2.0	3.0
Task Movement	2.5	2.0	2.7	3.0	3.0	3.0
Loop Addition	2.5	3.0	2.0	–	–	–
Flow Addition	–	3.0	2.0	–	–	–
Flow Deletion	–	3.0	3.0	–	–	–
Combined Changes	2.5	3.0	2.4	2.0	2.0	3.0

Table 4.4: Style evaluation scores by transformation type across all BPMN models (rounded to one decimal place).

Stylistic Score

Table 4.4 presents the style evaluation scores for each transformation type across six BPMN models. The same model labels are used as in the semantic evaluation: *SmallU*, *MediumU*, and *BigU* refer to small, medium, and large unbalanced models, while *SmallB*, *MediumB*, and *BigB* denote the balanced counterparts. The values represent single test runs per transformation and model, rather than averages. Scores range from 1 (stylistically inconsistent or intrusive) to 3 (fluent, concise, and closely matching the original tone), as defined in Table 4.1. A dash (–) indicates that the transformation was not evaluated for that model, either due to structural irrelevance or redundancy with prior tests.

The style evaluation results reveal several key trends. Most transformation types received high scores across both balanced and unbalanced models, indicating that the prompting strategy effectively preserves the tone and phrasing of the original process descriptions. Task deletions performed best, with consistently perfect scores and no stylistic degradation observed. Task additions followed closely, also showing strong results across all models, with only minor drops in large balanced cases. Task replacements showed slightly more variability, particularly in medium-sized models, where the phrasing sometimes drifted from the original tone. Task movements were the most stylistically challenging, especially in unbalanced models, likely due to irregular or ambiguous insertion contexts. Loop-related edits and flow manipulations also showed occasional inconsistencies, often caused by abrupt transitions introduced in the underlying structure. Combined transformations preserved style well in most cases, although some deviations appeared in complex scenarios. It is important to emphasize, however, that stylistic quality is only meaningful when semantic correctness is preserved. A fluent but misleading description can be more problematic than a slightly awkward but accurate one. As such, style scores should be interpreted in close conjunction with semantic evaluations.

Structural Alignment Score (SAS)

Table 4.5 reports the Structural Alignment Score (SAS) for each transformation type and BPMN model. These values reflect the alignment between machine-generated and human-edited texts, based on the metric defined in Section 4.2.4, which balances content preservation and deletion fidelity under a bag-of-words assumption. Since each score corresponds to a single representative test instance, the results should be viewed as qualitative indicators rather than statistically robust measurements.

Transformation Type	SmallU	MediumU	BigU	SmallB	MediumB	BigB
Task Addition	0.99	0.96	1.00	0.98	1.00	0.89
Task Deletion	0.89	1.00	0.62	0.86	0.93	0.89
Task Replacement	0.77	0.65	0.64	0.37	0.76	0.84
Task Movement	0.71	0.88	0.83	0.90	0.83	0.82
Loop Addition	0.99	1.00	0.85	—	—	—
Flow Addition	—	0.99	0.89	—	—	—
Flow Deletion	—	1.00	0.51	—	—	—
Combined Changes	0.62	0.95	0.72	0.46	0.88	0.87

Table 4.5: Structural Alignment Score (SAS) across transformation types and BPMN model variants (rounded to two decimal places).

Task additions consistently achieved near-perfect scores across all models, suggesting that the system tends to preserve existing content while integrating new elements with minimal disruption. Task deletions also performed strongly in most cases, though the relatively low score for the large unbalanced model (*BigU*, 0.62) could reflect greater alignment difficulty in structurally complex scenarios. However, this observation is based on a single instance and requires further validation.

Task replacements resulted in lower scores overall, particularly in smaller models such as *SmallB* (0.37). This indicates that replacement operations introduce more extensive structural divergence from the human reference, even when semantically correct. Compared to other transformation types, replacements appear more challenging for the system to express in a minimally edited form. While human editors often modify only a word or two—such as changing a verb or task label—the system must remove and regenerate the entire task description, including its control-flow context. This leads to greater surface-level token differences and consequently lower SAS scores.

Task movements yielded moderate to high SAS values, ranging from 0.71 to 0.90, with little variation between balanced and unbalanced processes. Loop additions and flow-related edits also showed high alignment, particularly in small and medium models. Flow deletion in the large unbalanced model scored significantly lower (0.51), indicating possible challenges in deletion precision under more irregular structures.

Combined transformations produced the most variation, with scores ranging from 0.46 to 0.95. While *SmallB* yielded the lowest result, larger models such as *BigB* (0.87) suggest that the method may remain effective even when multiple structural edits are applied simultaneously. Still, broader evaluation is needed to confirm this potential.

Execution Time

Table 4.6 reports the execution times (in seconds) for both the structural transformation step (*Search*) and the GPT-based text generation step (*Prompt*) for each transformation type across all BPMN models. The values correspond to single representative test runs per model and transformation type, rather than averaged results. The models are categorized by structure and size: *SmallU*, *MediumU*, and *BigU* refer to unbalanced models, while *SmallB*, *MediumB*, and *BigB* represent their balanced counterparts. The *Search* column indicates the time required to compute the transformation steps using either beam search (for unbalanced models) or the LCES algorithm (for balanced models). The *Prompt* column captures the total time for generating the updated textual description via GPT,

Transformation Type	SmallU		MediumU		BigU	
	Search	Prompt	Search	Prompt	Search	Prompt
Task Addition	1.68	5.11	5.85	6.75	7.52	13.76
Task Deletion	1.78	7.23	0.02	13.67	4.65	7.87
Task Replacement	0.01	9.01	0.02	7.65	0.02	17.87
Task Movement	4.76	13.50	9.62	14.67	30.33	22.99
Loop Addition	1.69	3.83	3.06	2.59	7.40	3.96
Flow Addition	–	–	3.08	3.71	16.71	4.88
Flow Deletion	–	–	5.24	3.13	14.84	3.59
Combined Changes	3.96	22.88	4.77	17.66	19.72	26.01
	SmallB		MediumB		BigB	
	Search	Prompt	Search	Prompt	Search	Prompt
Task Addition	0.02	6.02	0.05	4.56	0.03	8.95
Task Deletion	0.01	5.04	0.02	5.66	0.03	13.54
Task Replacement	0.02	4.11	0.04	11.19	0.04	14.94
Task Movement	0.02	6.36	0.05	13.58	0.04	24.01
Combined Changes	0.02	9.04	0.05	13.77	0.03	22.03

Table 4.6: Search and prompting times (in seconds) for each transformation across unbalanced and balanced BPMN models (rounded to two decimal places).

including all incremental prompting steps. A dash (–) indicates that the corresponding transformation was not applied to that model. All measurements were taken on a laptop equipped with an Intel Core i7-8565U CPU (1.80 GHz, 4 cores, 8 threads) and 16 GB of RAM. GPT prompting was performed via OpenAI’s API using the GPT-4.1 model.

The timing results highlight distinct performance characteristics between balanced and unbalanced models. For balanced BPMNs, the structural transformation step is nearly instantaneous across all transformation types and model sizes, as the LCES algorithm operates efficiently under structural assumptions such as balance and acyclicity. As a result, the total processing time is overwhelmingly dominated by the prompting phase, with durations ranging from approximately 4 to 24 seconds depending on the model size and transformation complexity. In contrast, unbalanced models show greater variation in search time as model size increases, due to the reliance on beam search. For small unbalanced models, prompting time still dominates, but in medium and large cases, the search phase becomes equally or even more expensive than the prompting phase. This is particularly evident in task movements and combined changes, where the number of possible alignments grows with model size. An exception occurs in some cases, such as task replacement in unbalanced models, where the search time is negligible. This happens when the original and modified BPMNs are found to be equivalent after the alignment phase, eliminating the need for any further transformation steps.

4.2.6 Discussion and Limitations

Structural and Semantic Limitations

Although the experimental results demonstrate that the method performs reliably on a variety of transformation types and model structures, there are several limitations worth noting. First, the evaluated transformations were deliberately limited in scope, typically involving only two to three structural edits per case. While this reflects common process update scenarios, larger-scale or cascading changes were not tested. In such cases, particularly in unbalanced models, the beam search strategy used for alignment may no longer converge reliably or within acceptable time limits—despite performing well across all tested instances.

Second, certain structural constructs posed specific difficulties. One such case involves exclusive gateways with “empty” branches—i.e., where one path has no tasks and effectively allows the flow to skip work. These constructs introduce semantic ambiguity, and the current transformation logic did not consistently handle them correctly in the regenerated text. Similar issues arise with loop constructs: although basic loop additions were processed syntactically, the system lacks explicit support for identifying or preserving loop semantics.

Third, while the method works well when the original process description is logical and clearly written, it is inherently limited by the quality of that input. If the original text is incomplete, ambiguous, or inconsistent with the BPMN structure, both the structural transformation and the resulting description may degrade in accuracy and coherence. This limits the system’s robustness in real-world scenarios where documentation quality may vary significantly.

A further limitation arises when newly added tasks are inserted into the model without sufficient surrounding context. If a newly inserted task depends on other tasks that have not yet been added, the resulting description can be misleading or semantically incoherent. In balanced, acyclic models, this can be mitigated by introducing changes in a breadth-first manner, allowing for predictable insertion ordering. However, in beam search scenarios—typical for unbalanced or cyclic processes—no such global ordering constraint can be enforced, making the update process more brittle when constructing entirely new control-flow segments.

SAS Behavior and Evaluation Limitations

The Structural Alignment Score (SAS) offers an interpretable estimate of how closely machine-generated revisions align with human edits, focusing on two key behaviors: retention of validated content and accurate deletion. It favors conservative editing and penalizes over- or under-editing relative to the reference. While it does not capture semantic nuance or style, it reliably reflects how well the system preserves and prunes content in line with human revision patterns.

However, several limitations affect how these scores should be interpreted. First, all metric results are based on single representative evaluations per transformation and model. While sufficient to illustrate behavioral trends, they do not provide statistical confidence or capture variability across multiple runs or examples.

Second, the metric operates under a bag-of-words assumption, ignoring word order, syntactic structure, and semantic equivalence. While this enables structural comparison without full parsing or alignment, it also penalizes valid rephrasings or necessary reformulations. This is particularly relevant in structurally motivated edits, where content may be split, reordered, or replaced in ways that are semantically appropriate but yield low surface-level overlap.

Third, the preservation ratio component is sensitive to input length. In longer texts, large portions of unchanged content can inflate preservation scores, even when important omissions occur. This can lead to an overly optimistic assessment of alignment in lengthy descriptions and obscure localized failures.

Future refinements could address these issues by incorporating token-level weighting (e.g., using inverse document frequency or domain relevance), computing preservation over syntactic or semantic segments, or learning expected preservation behavior based on input length. Embedding-based similarity metrics or hybrid approaches that combine structural and semantic cues could also complement the current metric. While not necessary at the current scale, such enhancements would be valuable for large-scale or real-world deployments where document variation is higher and fidelity requirements are stricter.

Performance and Execution Time

Execution time also presents a practical limitation, especially for real-time or large-scale applications. While performance was acceptable in the tested scenarios, the beam search phase in unbalanced models showed significant variability, with search times increasing noticeably for larger models and complex transformations. This reflects the algorithmic sensitivity of search-based alignment to model size and structural complexity, and can limit responsiveness in more demanding settings.

It is also important to note that the current system is a prototype and has not been optimized for performance. On one hand, redundant or non-essential computations remain in the current pipeline and could be removed to reduce unnecessary overhead. On the other hand, performance could also be improved independently through engineering enhancements such as multithreading, heuristic pruning, or memoization. These avenues are complementary and suggest that the system’s responsiveness could be significantly improved without fundamental changes to the underlying method.

More critically, in scenarios involving a large number of changes or structurally complex modifications, the search process may fail to find a valid alignment, resulting in no output. This reflects a practical limitation of the current method when handling extensive edits, particularly in larger or more irregular models.

Taken together, these observations suggest that while the approach is effective for small-scale, well-structured updates on realistic process models, its reliability may decline under more complex or inconsistent conditions. Future improvements could address these limitations by incorporating stronger structural validation, explicit loop modeling, and enhanced handling of semantic edge cases.

Related work

5.1 BPMN and UML Model Verbalization

A substantial body of work addresses the automatic generation of natural language (NL) descriptions from process and software models, aiming to make structured representations more accessible to non-technical stakeholders. In the context of BPMN models, Leopold et al. [13] introduced one of the earliest pipelines for transforming process models into natural language. Their approach decomposes BPMN models into structured fragments and maps them to sentences using a rule-based template system informed by linguistic theories. This was later extended in [14] to improve support for model validation by domain experts, emphasizing comprehensibility and linguistic completeness. Despite producing reliable output, these systems regenerate the entire description from scratch after every model change, which limits adaptability and introduces overhead when handling incremental updates.

Azevedo et al. [3] proposed a bidirectional synchronization framework between BPMN diagrams and accompanying textual instructions. While technically effective, it relies on full regeneration strategies and will be further discussed in the context of update-aware systems (see Section 5.2).

Similarly, the Process-to-Text (P2T) framework [8] generates summaries of mined process models using fuzzy linguistic techniques. Although not targeted at documentation, its summarization paradigm is relevant to evolving-process scenarios (see Section 5.2).

In the domain of software modeling, Burden and Heldal [6] introduced a system for verbalizing UML class diagrams by transforming them into an intermediate representation in Grammatical Framework (GF) before rendering them in controlled natural language. This approach ensures traceability and syntactic clarity, especially for safety-critical or regulated domains. Meziane et al. [17] built on this idea by incorporating WordNet-based disambiguation into their rule engine, enhancing semantic consistency in the generated specifications. However, both approaches require tightly controlled vocabularies and user discipline when naming elements in UML models, limiting scalability in freeform or agile environments.

Across these efforts, the predominant emphasis has been on generating fluent initial descriptions from models, typically using template-based or grammar-driven methods. However, none of these systems explicitly support incremental editing or semantic tracking of changes in evolving model-text pairs. Our work addresses this gap by enabling targeted, minimal updates to existing documentation in response to detected model changes—ensuring consistency while preserving fluency and authorial style.

5.2 Incremental and Update-Aware Documentation

While previous research has primarily focused on generating full textual representations from models, considerably less attention has been paid to the challenge of maintaining documentation in response to incremental model changes.

One of the few notable contributions in this area is the work by Zhu et al. [27], who introduce Business Process Text Sketches (BPTSs) derived from Conditional Process Trees using Large Language Models (LLMs). Their approach emphasizes abstract, semantically grounded summaries to facilitate dataset bootstrapping for downstream NLP tasks. While not designed to maintain or incrementally revise existing documentation, their use of versioned model transformations reflects a latent concern with model evolution. However, each change results in a freshly generated sketch, discarding prior phrasing, structural coherence, and contextual references.

Azevedo et al. [3], as noted earlier, support bidirectional synchronization between BPMN models and textual descriptions. Their system enables incremental propagation of changes from text to model; however, in the reverse direction, model updates trigger full regeneration of the associated documentation using templates. This ensures structural consistency but fails to preserve authorial style, contextual phrasing, or historical coherence.

The Process-to-Text (P2T) framework by Fontenla-Seco et al. [8] also engages with evolving process data. It generates high-level, time-sensitive textual summaries from mined process logs using linguistic protoforms and fuzzy quantifiers. While P2T supports incremental summarization across process executions, it does not maintain authored documentation or track sentence-level continuity.

In contrast, our approach introduces a change-aware documentation mechanism that integrates symbolic diffing techniques—such as Longest Common Execution Subsequence (LCES) and beam-aligned model traversal—with precision-targeted LLM prompting. This allows for accurate localization of changes within the model, mapping them to corresponding segments in the documentation, and generating only the necessary textual revisions. The result is a more efficient and stylistically consistent revision process, avoiding the drawbacks of full regeneration.

By treating documentation as a dynamic artifact that evolves in parallel with the underlying model, our method aligns with the iterative practices of agile development environments, where small, frequent changes are common and manual upkeep is often impractical.

5.3 Graph-to-Text Generation

Viewed abstractly, BPMN and UML models are structured graphs—comprising labeled nodes and typed edges—making the task of graph-to-text generation highly relevant to our problem setting. In this broader field, the challenge lies in capturing the rich relational structure of graphs while producing fluent and semantically faithful natural language output.

Several methods tackle this problem through architectural enhancements to language models. Zhao et al. [26] introduce a structure-aware Transformer that incorporates relative positional encoding into the attention mechanism, allowing the model to distinguish between first-order and more distant neighbors in a knowledge graph. Their approach improves factual grounding and topological coherence in the generated text but assumes the entire graph is static and available at generation time.

Li et al. [15] explore few-shot graph-to-text generation using large pretrained language models, demonstrating that structural knowledge can be induced from limited examples via prompt-based conditioning. This line of work shows promise for adapting to novel domains without finetuning, but does not address how to maintain or revise existing text when the graph changes incrementally.

Instruction-tuned models like GraphGPT [22] push the boundary further by grounding textual and graph representations jointly. GraphGPT introduces a text-graph alignment mechanism and dual-stage instruction tuning to enhance the model’s zero-shot reasoning over structural data. While powerful for single-pass tasks, these models are not designed to preserve narrative continuity or support revision workflows where the structure evolves over time.

Most of these approaches focus on one-shot generation: given a complete graph, produce a new textual summary. In contrast, our task requires *edit-aware generation*—not just producing text from structure, but *updating existing text to reflect structural deltas*. This introduces unique requirements: semantic alignment between old and new structures, preservation of phrasing and discourse style, and localization of edits.

Yuan and Färber [24] survey and benchmark graph-to-text systems, concluding that most current models prioritize one-shot fluency and overlook revision scenarios and sentence-level alignment. Additionally, while vision-driven pipelines like GenFlowchart [2] explore structured-to-text mappings in graphical domains, they similarly regenerate entire narratives from scratch rather than enabling revision-centric workflows.

To our knowledge, no current graph-to-text system explicitly addresses this revision-oriented setting, which positions our work at the intersection of symbolic diffing and guided natural language editing.

5.4 Prompt-Based Language Models for Structured Input

Prompt-based strategies have become increasingly central in adapting large language models (LLMs) to tasks involving structured input, including code, tables, and diagrams. These techniques leverage the generative capabilities of pretrained models without requiring task-specific finetuning, making them attractive for rapidly evolving domains.

Li et al. [15] demonstrate that even complex knowledge graphs can be used as input for text generation through prompt engineering alone. By crafting structured prompts that describe node-edge relationships, they show that LLMs can generalize across domains in few-shot and zero-shot setups. Such strategies highlight the ability of LLMs to reason over structured data using only textual interfaces.

Another approach, Code2Seq [1], encodes abstract syntax trees as paths and generates summaries using sequence models. Although not prompt-based in the modern LLM sense, this method inspired subsequent research on representing structured models in a linearized form suitable for neural generation.

While these systems have demonstrated that structural biases can be injected via prompts or path encodings, they are typically optimized for full text generation from static inputs. Our approach instead treats the prompt as a mechanism for *targeted revision*, encoding not only structural elements but also the *difference* between an earlier and modified version of a model. This enables localized prompting: instead of regenerating entire documents, we prompt the LLM to revise only the sentences affected by the change. This hybrid use of symbolic diffs and natural language instructions introduces a novel interaction pattern that better suits agile and collaborative documentation workflows.

Other multimodal approaches, such as GenFlowchart [2], also employ prompt engineering with OCR-extracted structure to guide generative models. However, these systems focus on single-pass reconstruction rather than maintaining narrative continuity through targeted revisions.

5.5 Summary and Positioning

Approaches to model-to-text generation span a spectrum from rigid, template-based systems to flexible, one-shot neural generators. Template-driven methods offer structural reliability and repeatability but struggle with fluency, adaptability, and preservation of authorial nuance. Neural models—particularly those based on pretrained language models—achieve fluent, contextually rich output but typically assume static inputs and regenerate text from scratch upon each structural change.

Prompt-based techniques strike a promising balance by enabling structured inputs to guide generative models without requiring task-specific finetuning. They have demonstrated effectiveness in one-shot generation across various structured domains, including code, tables, and knowledge graphs. However, these methods predominantly target full-sample generation and do not support localized updates or track semantic continuity through evolving structures.

Across the literature, the problem of incrementally updating existing documentation in response to small, targeted changes in structured models remains largely unaddressed. Most existing systems regenerate text from scratch or disregard prior linguistic context, which disrupts narrative continuity and makes maintaining stylistic coherence difficult in evolving environments.

Our approach addresses this gap by integrating symbolic diffing techniques with prompt-guided language model updates. Rather than regenerating entire documents, we detect structural deltas and guide edits to the affected portions of the documentation. This enables revisions that preserve prior phrasing and narrative structure, supporting semantic consistency and stylistic continuity. The result is a revision-oriented workflow that treats documentation as a dynamic artifact evolving alongside the model—an essential capability in agile and collaborative settings.

Conclusion

This thesis addressed the challenge of keeping textual process descriptions aligned with evolving BPMN models. As business processes change—through task insertions, deletions, or reordering—it becomes essential to update documentation accurately without rewriting it entirely.

To this end, a structured methodology was introduced for deriving transformation steps: a sequence of localized edits that map the differences between the original and modified BPMN models. These steps serve as both a minimal set of structural changes and a precise blueprint for guiding textual updates. By anchoring modifications to these steps, the system preserves unchanged parts of the description while applying only necessary edits—maintaining stylistic consistency and structural coherence.

Two strategies were developed to compute transformation steps: the Longest Common Execution Sequence (LCES) method, which targets balanced and acyclic models, and a heuristic Beam Search strategy for models with loops or unbalanced control flow.

A GPT-based prompting framework operationalizes these transformation steps, incrementally guiding the language model to apply each change while preserving semantic fidelity and stylistic alignment with the original text.

Experimental results show that the system produces updates with high semantic and stylistic consistency, as measured by automated metrics. However, its effectiveness tends to decline with large-scale model changes—such as major insertions, deletions, or structural shifts—where maintaining coherence across many consecutive edits becomes more difficult.

Overall, this work offers a principled and scalable approach to incremental process documentation. By combining structural alignment with prompt-driven generation, it enables minimally invasive updates that adapt fluidly to evolving BPMN models.

Directions for Future Work

Several directions remain to extend and refine this work.

The Beam Search component currently employs a simple cost heuristic based on edit operations. Future enhancements could incorporate more sophisticated measures of structural similarity—such as graph kernels, embeddings, or BPMN-specific fingerprints—which would enable more semantically aware search behavior. A richer cost model could further prioritize meaningful changes by considering node types, control-flow roles, or execution impact.

Support for group-level transformations could be added to improve handling of changes affecting entire branches or sub-processes. This would reduce the number of transformation steps and better preserve logical groupings in the process.

Another avenue involves pattern-based prompting. By recognizing common BPMN substructures—like loops, parallel flows, or decision points—the system could apply tailored prompts, improving both generation accuracy and contextual awareness.

The natural language generation component may also benefit from domain-specific fine-tuning. Training the language model on real-world process documentation could improve fluency, terminology, and style adherence, reducing reliance on handcrafted prompts.

Beyond actor preservation, richer contextual reconstruction could be explored. This includes capturing tone, formality, temporal expressions, or narrative voice—elements critical for producing high-quality documentation.

Lastly, broader support for BPMN 2.0 elements—including conditional, inclusive, and event-based gateways, as well as pools and lanes—would expand the applicability of the system and bring it closer to comprehensive process documentation support.

Collectively, these directions point to the potential for building more robust, adaptive, and semantically aware systems for maintaining high-quality textual documentation of dynamic business processes.

— A —

LCES Pseudocode

The following pseudocode provides a high-level, language-agnostic implementation of the Longest Common Execution Sequence (LCES) algorithm for process model alignment. Its structure and naming directly reflect the recursive definitions and semantic considerations detailed in Section 3.2.3.

The primary entry point is the LCES procedure, which takes a pair of nodes from the respective BPMN process graphs and computes all longest common execution subsequences of matched node pairs, while preserving the control-flow and structural properties of both models. The algorithm leverages memoization to avoid redundant computation and systematically explores all possibilities: matching the current node pair, skipping a node in either model, and recursively combining the resulting alignments.

Special handling is provided for matched split gateways (e.g., parallel or exclusive branches): the algorithm computes branch-wise LCES alignments and then applies a dedicated flattening procedure, `FLATTEN`, to construct semantically consistent alignments across corresponding branches. The `MAXIMALLYALIGNEDBRANCHES` routine identifies all maximally aligned and pairwise-distinct branch combinations, and the fallback mechanism (as described in Definition 15) ensures that longer, unstructured branch-level alignments are retained whenever they yield a longer common subsequence.

Auxiliary procedures such as `UPDATEBEST` manage the sets of maximal-length solutions throughout the recursion, ensuring that all optimal alignments are collected. All pseudocode is structured for clarity and direct correspondence with the formal definitions in the main text, with descriptive variable names and stepwise control flow.

This implementation serves as a reference for both conceptual understanding and practical realization of LCES computation in the context of process model comparison.

Algorithm 1 $\text{LCES}(v, v')$

Require: $v \in V, v' \in V'$ **Ensure:** Longest path length ℓ and corresponding set of paths $\mathcal{C}$

```
1: if  $(v, v') \in \text{memo}$  then                                ▷ Check if result already memoized
2:   return  $\text{memo}[(v, v')]$ 
3: end if
4:  $\ell \leftarrow 0, \mathcal{C} \leftarrow \emptyset$                                 ▷ Initialize path length and set of continuations
5: if  $\text{MATCH}(v, v')$  then                                ▷ Check if current nodes are semantically or structurally matched
6:    $\text{branchCombinations} \leftarrow \emptyset$ 
7:   for all  $u \in \text{NEXT}(v)$  do                                ▷ Iterate over all successors of  $v$ 
8:     for all  $u' \in \text{NEXT}(v')$  do                                ▷ Iterate over all successors of  $v'$ 
9:        $(\ell', \mathcal{S}) \leftarrow \text{LCES}(u, u')$                                 ▷ Recursively compute LCES for successors
10:       $\text{UPDATEBEST}(\ell, \mathcal{C}, \ell', \mathcal{S})$                                 ▷ Update best path set if longer found
11:      if  $\text{ISPLIT}(v)$  and  $\text{ISPLIT}(v')$  then                                ▷ If both are split gateways
12:         $\text{branchCombinations}[(u, u')] \leftarrow \mathcal{S}$                                 ▷ Collect branch-wise results
13:      end if
14:    end for
15:  end for
16:  if  $\text{ISPLIT}(v)$  and  $\text{ISPLIT}(v')$  then                                ▷ If both are split gateways
17:     $\mathcal{F} \leftarrow \text{FLATTEN}(\text{branchCombinations}, (v, v'))$                                 ▷ Flatten branches and handle fallback
18:    if  $\mathcal{F} \neq \emptyset$  then                                ▷ If flattening yields valid alignments
19:       $\ell'' \leftarrow \max\{|\pi| \mid \pi \in \mathcal{F}\}$                                 ▷ Get maximal length after flattening
20:       $\text{UPDATEBEST}(\ell, \mathcal{C}, \ell'', \mathcal{F})$                                 ▷ Update if flattened region is longer
21:    end if
22:  end if
23:   $\ell \leftarrow \ell + 1$                                 ▷ Include the current matching pair in path length
24:  if  $\mathcal{C} = \emptyset$  then
25:     $\mathcal{C} \leftarrow \{(v, v')\}$                                 ▷ Initialize new path starting at  $(v, v')$ 
26:  else
27:     $\mathcal{C} \leftarrow \{(v, v')\} \cup \{\pi \mid \pi \in \mathcal{C}\}$                                 ▷ Prepend current pair to all continuations
28:  end if
29: end if
30: for all  $u \in \text{NEXT}(v)$  do                                ▷ Option: skip  $v$  and continue in  $V$  only
31:    $(\ell', \mathcal{S}) \leftarrow \text{LCES}(u, v')$ 
32:    $\text{UPDATEBEST}(\ell, \mathcal{C}, \ell', \mathcal{S})$ 
33: end for
34: for all  $u' \in \text{NEXT}(v')$  do                                ▷ Option: skip  $v'$  and continue in  $V'$  only
35:    $(\ell', \mathcal{S}) \leftarrow \text{LCES}(v, u')$ 
36:    $\text{UPDATEBEST}(\ell, \mathcal{C}, \ell', \mathcal{S})$ 
37: end for
38:  $\text{memo}[(v, v')] \leftarrow (\ell, \mathcal{C})$                                 ▷ Memoize result for future calls
39: return  $(\ell, \mathcal{C})$                                 ▷ Return maximal length and corresponding paths
```

Algorithm 2 FLATTEN($childPairExecutionPaths, (v, v')$)

Require: $childPairExecutionPaths: (u, u') \mapsto$ set of paths for each branch pair

1: (v, v') : current matched split gateway pair

Ensure: $recombinedPaths$: set of longest recombined (flattened) paths

2: $m \leftarrow mergeMapping[v]$

▷ Get corresponding merge node for v

3: $m' \leftarrow mergeMapping'[v']$

▷ Get corresponding merge node for v'

4: $mergeReachingPaths \leftarrow \emptyset$

▷ Store pre-merge paths for each branch pair

5: $postMergeSegments \leftarrow \emptyset$

▷ Store post-merge path segments

6: **for all** $(u, u') \mapsto \mathcal{P}$ in $childPairExecutionPaths$ **do**

7: $\mathcal{P}_{pre} \leftarrow \emptyset$

8: **for all** $\pi \in \mathcal{P}$ **do**

▷ Iterate over all paths for the branch pair

9: $\pi_{pre} \leftarrow \emptyset, \pi_{post} \leftarrow \emptyset$

10: $mergeReached \leftarrow \text{false}$

11: **for all** $(x, y) \in \pi$ **do**

▷ Split path at first merge node pair

12: **if not** $mergeReached$ **then**

13: Append (x, y) to π_{pre}

14: **if** $x = m$ **and** $y = m'$ **then**

15: $mergeReached \leftarrow \text{true}$

16: **end if**

17: **else**

18: Append (x, y) to π_{post}

19: **end if**

20: **end for**

21: **if** $\pi_{pre} \neq \emptyset$ **and** $mergeReached$ **then**

22: $\mathcal{P}_{pre} \leftarrow \mathcal{P}_{pre} \cup \{\pi_{pre}\}$

▷ Keep prefix only if merge was reached

23: **end if**

24: **if** $\pi_{post} \neq \emptyset$ **then**

25: $postMergeSegments \leftarrow postMergeSegments \cup \{\pi_{post}\}$

▷

Collect all post-merge suffixes

26: **end if**

27: **end for**

28: **if** $\mathcal{P}_{pre} \neq \emptyset$ **then**

29: $mergeReachingPaths[(u, u')] \leftarrow \mathcal{P}_{pre}$

30: **end if**

31: **end for**

32: $\mathcal{M} \leftarrow \text{MAXIMALLYALIGNEDBRANCHES}(mergeReachingPaths)$

▷

Find all maximally aligned, pairwise-distinct pre-merge combinations

33: $L \leftarrow \max\{|\pi| \mid \pi \in \mathcal{M}\}$ **or** 0

34: $\mathcal{M}_{max} \leftarrow \{\pi \in \mathcal{M} \mid |\pi| = L\}$

▷ Select only longest combinations

35: $L_o \leftarrow \max\{|\sigma| \mid \sigma \in postMergeSegments\}$ **or** 0

36: $\mathcal{O}_{max} \leftarrow \{\sigma \in postMergeSegments \mid |\sigma| = L_o\}$

▷ Longest post-merge suffixes

37: $recombinedPaths \leftarrow \emptyset$

38: **for all** $\pi \in \mathcal{M}_{max}$ **do**

39: **for all** $\sigma \in \mathcal{O}_{max}$ **do**

40: $recombinedPaths \leftarrow recombinedPaths \cup \{\pi \cup \sigma\}$

▷

Concatenate each pre-merge and post-merge segment

41: **end for**

42: **end for**

43: **return** $recombinedPaths$

▷ Return all valid flattened path combinations

Algorithm 3 MAXIMALLYALIGNEDBRANCHES($mergeReachingPaths$)

Require: $mergeReachingPaths$: map $(u, u') \mapsto$ set of pre-merge paths for each branch pair

Ensure: $\mathcal{M}$: set of maximally aligned, pairwise-distinct path combinations

```
1: Let  $\mathcal{K} \leftarrow$  list of keys in  $mergeReachingPaths$   $\triangleright$  All entry node pairs  $(u, u')$ 
2:  $\mathcal{M} \leftarrow \emptyset$   $\triangleright$  Initialize the set of valid branch combinations
3: BACKTRACK(0,  $\emptyset$ ,  $\emptyset$ ,  $\emptyset$ )  $\triangleright$  Start recursive search
4: return  $\mathcal{M}$ 
5: function BACKTRACK( $idx, used_1, used_2, selected$ )  $\triangleright$  Store the current valid combination
6:   Add a copy of  $selected$  to  $\mathcal{M}$ 
7:   for  $i \leftarrow idx$  to  $|\mathcal{K}| - 1$  do
8:      $(u, u') \leftarrow \mathcal{K}[i]$   $\triangleright$  Consider next branch pair
9:     if  $u \notin used_1$  and  $u' \notin used_2$  then  $\triangleright$  Ensure pairwise-distinct branch mapping
10:      for all path  $\pi \in mergeReachingPaths[(u, u')]$  do
11:        Append  $\pi$  to  $selected$   $\triangleright$  Add path for current branch pair
12:        Add  $u$  to  $used_1$ ; Add  $u'$  to  $used_2$   $\triangleright$  Mark branches as used
13:        BACKTRACK( $i + 1, used_1, used_2, selected$ )  $\triangleright$  Recurse with next index
14:        Remove last element from  $selected$   $\triangleright$  Backtrack: remove current path
15:        Remove  $u$  from  $used_1$ ; Remove  $u'$  from  $used_2$   $\triangleright$  Backtrack: unmark branches
16:      end for
17:    end if
18:  end for
19: end function
```

Algorithm 4 UPDATEBEST($\ell, \mathcal{C}, \ell', \mathcal{S}$)

Require: ℓ : current maximal path length

1: $\mathcal{C}$: current set of maximal paths

2: ℓ' : candidate path length

3: $\mathcal{S}$: candidate set of paths

Ensure: Update ℓ and $\mathcal{C}$ if a longer or equally long path is found

```
4: if  $\ell' > \ell$  then
5:    $\ell \leftarrow \ell'; \mathcal{C} \leftarrow \mathcal{S}$   $\triangleright$  Replace current best if strictly longer
6: else if  $\ell' = \ell$  then
7:    $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{S}$   $\triangleright$  Add candidates if equal in length
8: end if
```

— B —

Tool Interface

This appendix provides screenshots of the web-based tool described in Section 4.1.3. The first figure shows the interface after generating the BPMN model from the initial textual description. The second figure shows the interface after the user has modified the BPMN model and triggered the transformation to update the textual description.

GIVUP: Generation and Verification of Underspecified Processes

Password:

Business process:

 Describe your BPMN

 Upload your BPMN

First of all, an employee collects goods.
 Then, the client pays for the delivery while the employee prepares the parcel.
 Finally, the company can either deliver by car or deliver by drone.

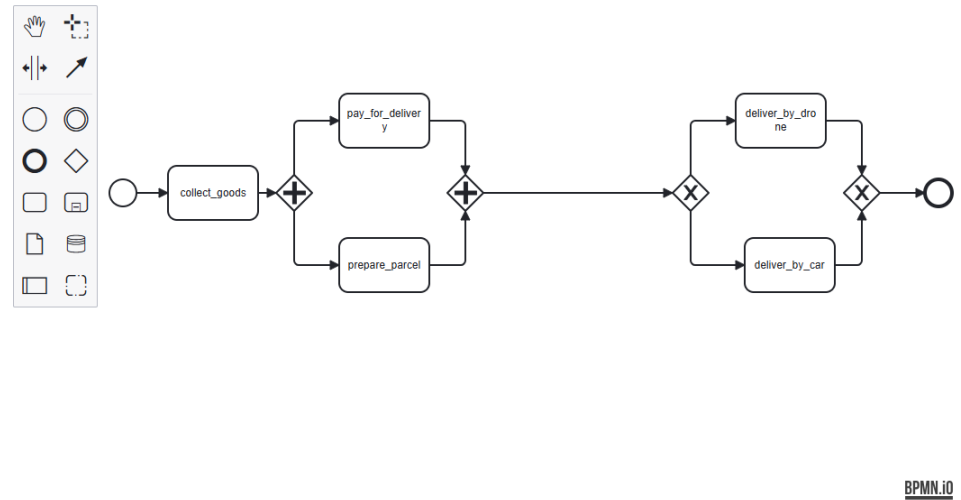
☒ Tasks already named

Submit

 Reset

 Adapt Description

 Download



BPMN.io

Figure B.1: Web interface after BPMN model generation from initial description

GIVUP: Generation and Verification of Underspecified Processes

Password:

Business process:

 Describe your BPMN

 Upload your BPMN

First of all, an employee collects goods.
 Then, the client pays for the delivery while the employee prepares the parcel.
 After both payment and preparation are complete, the company sends a receipt.
 Finally, the company can either deliver by car or deliver by drone.

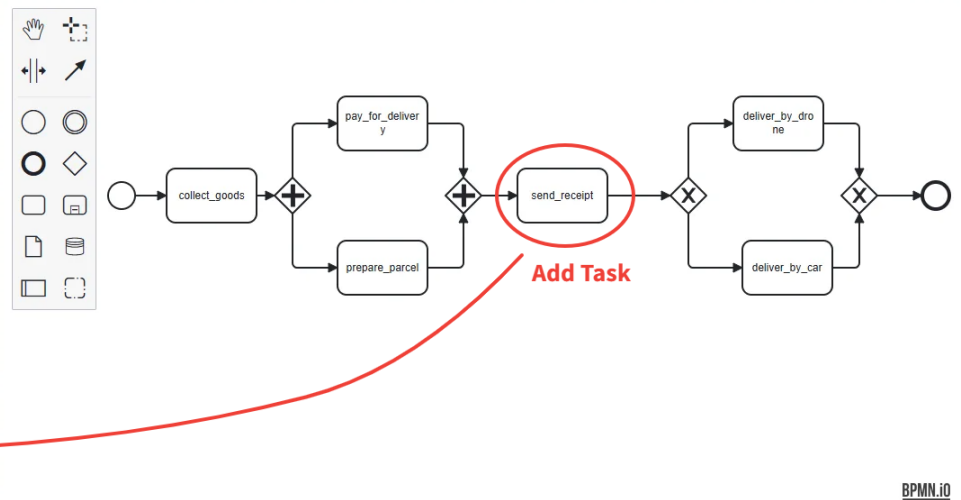
☒ Tasks already named

Submit

 Reset

 Adapt Description

 Download



BPMN.io

Figure B.2: Web interface after triggering the transformation with an updated BPMN model

Bibliography

- [1] Uri Alon, Shaked Brody, Omer Levy, and Eran Yahav. code2seq: Generating sequences from structured representations of code, 2019.
- [2] Abdul Arbaz, Heng Fan, Junhua Ding, Meikang Qiu, and Yunhe Feng. Genflowchart: Parsing and understanding flowchart using generative AI. In Cungeng Cao, Huajun Chen, Liang Zhao, Junaid Arshad, Taufiq Asyhari, and Yonghao Wang, editors, *Knowledge Science, Engineering and Management*, pages 99–111, Singapore, 2024. Springer Nature Singapore.
- [3] Leonardo Guerreiro Azevedo, Raphael de Almeida Rodrigues, and Kate Revoredo. BPMN model and text instructions automatic synchronization. In *Proceedings of the 20th International Conference on Enterprise Information Systems (ICEIS), Volume 1*, pages 484–491, 2018.
- [4] Patrizio Bellan, Han van der Aa, Mauro Dragoni, Chiara Ghidini, and Simone Paolo Ponzetto. Pet: An annotated dataset for process extraction from natural language text tasks. In Cristina Cabanillas, Niels Frederik Garmann-Johnsen, and Agnes Koschmider, editors, *Business Process Management Workshops*, pages 315–321, Cham, 2023. Springer International Publishing.
- [5] H. Bunke. On a relation between graph edit distance and maximum common subgraph. *Pattern Recognition Letters*, 18(8):689–694, 1997.
- [6] Håkan Burden and Rogardt Heldal. Natural language generation from class diagrams. In *Proceedings of the 8th International Workshop on Model-Driven Engineering, Verification and Validation (MoDeVVA)*, page 8, New York, NY, USA, 2011. Association for Computing Machinery.
- [7] Marlon Dumas, Marcello La Rosa, Jan Mendling, Hajo A Reijers, et al. *Fundamentals of business process management*, volume 1. Springer, 2013.
- [8] Yago Fontenla-Seco, Manuel Lama, and Alberto Bugarín. Process-to-text: A framework for the quantitative description of processes in natural language. In *Trustworthy AI - Integrating Learning, Optimization and Reasoning*, pages 212–219. Springer International Publishing, 2021.
- [9] F. Friedrich. Automated generation of business process models from natural language input. M.sc. thesis, School of Business and Economics, Humboldt-Universität zu Berlin, 2010.
- [10] D. S. Hirschberg. A linear space algorithm for computing maximal common subsequences. *Commun. ACM*, 18(6):341–343, June 1975.

- [11] International Organization for Standardization and International Electrotechnical Commission. Information technology — object management group business process model and notation. ISO/IEC 19510:2013, 2013. Based on OMG Specification BPMN 2.0.
- [12] Ajay Krishna, Pascal Poizat, and Gwen Salaün. Checking business process evolution. *Science of Computer Programming*, 170:1–26, 2019.
- [13] Henrik Leopold, Jan Mendling, and Artem Polyvyanyy. Generating natural language texts from business process models. In Jolita Ralyté, Xavier Franch, Sjaak Brinkkemper, and Stanislaw Wrycza, editors, *Advanced Information Systems Engineering*, pages 64–79, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [14] Henrik Leopold, Jan Mendling, and Artem Polyvyanyy. Supporting process model validation through natural language generation. *IEEE Transactions on Software Engineering*, 40(8):818–840, 2014.
- [15] Junyi Li, Tianyi Tang, Wayne Xin Zhao, Zhicheng Wei, Nicholas Jing Yuan, and Ji-Rong Wen. Few-shot knowledge graph-to-text generation with pretrained language models, 2021.
- [16] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain, July 2004. Association for Computational Linguistics.
- [17] Farid Meziane, Nikos Athanasakis, and Sophia Ananiadou. Generating natural language specifications from UML class diagrams. *Requirements Engineering*, 13(1):1–18, 2008.
- [18] Michel Neuhaus, Kaspar Riesen, and Horst Bunke. Fast suboptimal algorithms for the computation of graph edit distance. In Dit-Yan Yeung, James T. Kwok, Ana Fred, Fabio Roli, and Dick de Ridder, editors, *Structural, Syntactic, and Statistical Pattern Recognition*, pages 163–172, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
- [19] Quentin Nivon and Gwen Salaün. Automated generation of bpmn processes from textual requirements. In Walid Gaaloul, Michael Sheng, Qi Yu, and Sami Yanguis, editors, *Service-Oriented Computing*, pages 185–201, Singapore, 2025. Springer Nature Singapore.
- [20] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ACL ’02, pages 311–318, USA, 2002. Association for Computational Linguistics.
- [21] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(9), 2011.
- [22] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang. Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’24)*, pages 491–500, New York, NY, USA, 2024. Association for Computing Machinery.

- [23] Han Van der Aa, Henrik Leopold, and Hajo A Reijers. Detecting inconsistencies between process models and textual descriptions. In *Business Process Management: 13th International Conference, BPM 2015, Innsbruck, Austria, August 31–September 3, 2015, Proceedings 13*, pages 90–105. Springer, 2015.
- [24] Shuzhou Yuan and Michael Färber. Evaluating generative models for graph-to-text generation, 2023.
- [25] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert, 2020.
- [26] Feng Zhao, Hongzhi Zou, and Cheng Yan. Structure-aware knowledge graph-to-text generation with planning selection and similarity distinction. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8693–8703, Singapore, December 2023. Association for Computational Linguistics.
- [27] Rui Zhu, Quanzhou Hu, Wenxin Li, Honghao Xiao, Chaogang Wang, and Zixin Zhou. Business process text sketch automation generation using large language model, 2023.